

MICROSOFT CERTIFICATION — BOOK 2 OF 4

MASTERING AZ-104

Microsoft Azure Administrator

A complete, hands-on course — from zero to administering Azure

Every exam objective explained in plain English, following the official Microsoft study guide domain by domain, sub-group by sub-group, in the exact order Microsoft presents them

Aligned to the official skills measured as of 17 April 2026
EDITION 2.0 — realigned to the official study guide order

Preface: How to Use This Book

AZ-104 is a doing exam. This book teaches you the concepts and then shows you how each task is actually done — in the portal and on the command line — so the skills stick.

This edition is deliberately restructured to follow Microsoft's own study guide in the exact order Microsoft presents it — domain by domain, and within each domain, sub-group by sub-group. If you're studying with the official skills-measured outline open next to you, this book's chapters and sections track it one-to-one, so you never have to hunt for where a bullet point is covered.

The learning aids on every topic

IN PLAIN ENGLISH

Strips the jargon and says what a thing is in everyday words.

ANALOGY

Ties a technical idea to real life so it sticks.

COMMAND EXAMPLE

```
# The exact portal path or command to perform a task  
az group create --name myRG --location eastus
```

MEMORIZE THIS

The precise fact the exam tests — the numbers, limits, and distinctions.

EXAM TIP

The trick wordings and look-alikes that cost marks.

How to study for a hands-on exam

- **Read for understanding first**, then re-read the boxes and tables where the testable facts live.
- **Do the labs.** Get a free Azure account (\$200 credit for 30 days) and actually perform each task.
- **Learn portal AND CLI.** Questions come both ways. When you read a command box, run it.
- **Use the Quick Checks** at the end of each chapter, and the practice bank in the appendix.
- **Study alongside the official outline** — this book's section numbers follow it in order, so you can check items off as you go.

Contents

Preface: How to Use This Book	2
Orientation: The Azure Foundation	8
0.1 How Azure is organized: the management hierarchy	8
0.2 Regions and availability zones	8
0.3 Azure Resource Manager and your admin tools	9
0.4 About the AZ-104 exam	9
0.5 How AZ-104 questions actually get tricky	10
Foundations	12
Before the Objectives: Foundations	13
Foundations: Identity & Governance	14
F1.1 Why identity became the control point	14
F1.2 Why RBAC exists — and the one problem it deliberately does NOT solve	15
F1.3 Why Azure Policy exists as a SEPARATE system from RBAC	15
F1.4 Why the management hierarchy exists	16
F1.5 Putting it all together: one governance model, assembled	16
Foundations: Storage	17
F2.1 Why access tiers exist: a physical and economic tradeoff, not a preference	17
F2.2 Why redundancy comes in four flavors instead of one 'safe' option	17
F2.3 Why three different access methods exist for the same data	18
F2.4 Why blob protection is three separate features, not one	18
F2.5 Putting it all together: one storage design, assembled	18
Foundations: Compute	20
F3.1 Why infrastructure as code exists, and what it actually costs	20
F3.2 Why VMs, containers, and App Service are one spectrum, not three unrelated tools	20
F3.3 Why availability sets AND zones both exist	21
F3.4 Putting it all together: one compute design, assembled	21
Foundations: Networking	23
F4.1 Why networks are segmented instead of left flat	23
F4.2 Why NSGs exist as a LIMITED tool, on purpose	23
F4.3 Why Bastion exists as a dedicated service, not just an NSG rule	24
F4.4 Why hub-and-spoke became the standard pattern	24
F4.5 Putting it all together: one network design, assembled	24
Foundations: Monitoring & Recovery	26
F5.1 Why metrics and logs both exist instead of just one	26

F5.2 Why alerting exists, and what it deliberately doesn't guarantee	26
F5.3 Why backup and Site Recovery are two separate services	27
F5.4 Putting it all together: one operational readiness model, assembled	27

Identities & Governance 30

Microsoft Entra Users & Groups	31
1.1 Creating users and groups	31
1.2 Managing user and group properties	32
1.3 Managing licenses in Microsoft Entra ID	32
1.4 Managing external users	33
1.5 Configuring self-service password reset (SSPR)	33
1.6 Sharpen the edge: bulk operations and dynamic-group timing	34
1.7 Hands-on lab: create a dynamic group and license it	34
Azure RBAC: Access to Resources	36
2.1 How Azure RBAC works	36
2.2 Managing built-in Azure roles	36
2.3 Assigning roles at different scopes	37
2.4 Interpreting access assignments	37
2.5 Sharpen the edge: RBAC vs Entra roles	38
2.6 Hands-on lab: assign a scoped role and verify inheritance	38
Subscriptions & Governance	40
3.1 Implementing and managing Azure Policy	40
3.2 Configuring resource locks	41
3.3 Applying and managing tags	41
3.4 Managing resource groups	41
3.5 Managing subscriptions	41
3.6 Managing costs: alerts, budgets, and Azure Advisor	41
3.7 Configuring management groups	42
3.8 Hands-on lab: enforce tagging with Azure Policy	42

Storage 44

Configuring Access to Storage	45
4.1 Configuring Azure Storage firewalls and virtual networks	45
4.2 Creating and using shared access signature (SAS) tokens	45
4.3 Configuring stored access policies	46
4.4 Managing access keys	46
4.5 Configuring identity-based access for Azure Files	46
4.6 Hands-on lab: issue and test a service SAS	47
Storage Accounts & Redundancy	48

5.1 Creating and configuring storage accounts	48
5.2 Configuring Azure Storage redundancy	48
5.3 Configuring object replication	49
5.4 Configuring storage account encryption	49
5.5 Managing data with Storage Explorer and AzCopy	49
5.6 Hands-on lab: configure GRS and observe the failover process	49
Blob Storage & Azure Files	51
6.1 Creating and configuring a file share in Azure Files	51
6.2 Creating and configuring a container in Azure Blob Storage	51
6.3 Configuring storage tiers	51
6.4 Configuring soft delete for blobs and containers	52
6.5 Configuring snapshots and soft delete for Azure Files	52
6.6 Configuring blob lifecycle management	52
6.7 Configuring blob versioning	52
6.8 Hands-on lab: build a full data-protection stack on one container	52
Compute	54
Infrastructure as Code: ARM & Bicep	55
7.1 Interpreting an ARM template or Bicep file	55
7.2 Modifying an existing ARM template	55
7.3 Modifying an existing Bicep file	56
7.4 Deploying resources with an ARM template or Bicep file	56
7.5 Exporting a deployment, or converting ARM to Bicep	56
7.6 Hands-on lab: deploy, modify, and redeploy a Bicep file	56
Virtual Machines & Availability	58
8.1 Creating a virtual machine	58
8.2 Configuring encryption at host	58
8.3 Moving a virtual machine	59
8.4 Managing virtual machine sizes	59
8.5 Managing virtual machine disks	59
8.6 Deploying VMs to availability zones and availability sets	59
8.7 Deploying and configuring a Virtual Machine Scale Set	60
8.8 Hands-on lab: deploy two VMs into an availability set	60
Containers in Azure	62
9.1 Creating and managing Azure Container Registry	62
9.2 Provisioning a container with Azure Container Instances	62
9.3 Provisioning a container with Azure Container Apps	63
9.4 Managing sizing and scaling for containers	63
9.5 Hands-on lab: build and run a container end to end	63

Azure App Service	65
10.1 Provisioning an App Service plan	65
10.2 Configuring scaling for an App Service plan	65
10.3 Creating an App Service	65
10.4 Configuring certificates and TLS	65
10.5 Mapping a custom DNS name	66
10.6 Configuring backup for an App Service	66
10.7 Configuring networking settings	66
10.8 Configuring deployment slots	66
10.9 Hands-on lab: deploy an app with a staging slot and swap	66
Networking	68
Virtual Networks & Connectivity	69
11.1 Creating and configuring virtual networks and subnets	69
11.2 Creating and configuring virtual network peering	69
11.3 Configuring public IP addresses	69
11.4 Configuring user-defined routes	70
11.5 Troubleshooting network connectivity	70
11.6 Hands-on lab: peer two VNets and prove non-transitivity	70
Securing Access to Virtual Networks	72
12.1 Creating NSGs and application security groups	72
12.2 Evaluating effective security rules in NSGs	72
12.3 Implementing Azure Bastion	73
12.4 Configuring service endpoints for Azure PaaS	73
12.5 Configuring private endpoints for Azure PaaS	73
12.6 Hands-on lab: reach a VM via Bastion and a storage account via private endpoint	74
Name Resolution & Load Balancing	76
13.1 Configuring Azure DNS	76
13.2 Configuring an internal or public load balancer	76
13.3 Troubleshooting load balancing	77
13.4 Hands-on lab: build a load-balanced pair with a health probe	77
Monitoring & Maintenance	79
Monitoring with Azure Monitor	80
14.1 Interpreting metrics in Azure Monitor	80
14.2 Configuring log settings in Azure Monitor	80
14.3 Querying and analyzing logs in Azure Monitor	81
14.4 Setting up alert rules, action groups, and processing rules	81
14.5 Configuring and interpreting Azure Monitor Insights	81

14.6 Using Azure Network Watcher and Connection monitor	82
14.7 Hands-on lab: wire up a metric alert end to end	82
Backup & Recovery	84
15.1 Creating a Recovery Services vault	84
15.2 Creating an Azure Backup vault	84
15.3 Creating and configuring a backup policy	84
15.4 Performing backup and restore operations	85
15.5 Configuring Azure Site Recovery	85
15.6 Performing a failover to a secondary region	85
15.7 Configuring and interpreting reports and alerts for backups	85
15.8 Hands-on lab: back up a VM and test a file-level restore	86
 Troubleshooting Guide	 87
 What Works, What Doesn't: A Troubleshooting Reference	 88
Domain 1 — Identities & Governance	88
Domain 2 — Storage	88
Domain 3 — Compute	88
Domain 4 — Networking	89
Domain 5 — Monitoring & Maintenance	89
 Case Study Practice	 91
 Case Study Practice	 92
Case Study 1: Contoso Manufacturing	92
Case Study 2: Fabrikam Health	93
 Appendix	 95
 Appendix A · Glossary	 96
 Appendix B · Exam-Day Strategy	 98
 Appendix C · CLI & PowerShell Quick Reference	 99
 Appendix D · Practice Question Bank	 100

Orientation: The Azure Foundation

WHAT YOU'LL LEARN

- How Azure is organized: tenants, subscriptions, resource groups, resources
- Regions, availability zones, and why location matters
- Azure Resource Manager, the portal, CLI, PowerShell, and Cloud Shell
- What the AZ-104 exam is, how it's scored, and how this book maps to it

IN PLAIN ENGLISH

If you've never opened the Azure portal before, here's the one-sentence version of this whole book: Azure is a website full of "rent-a-computer-thing" services, and being an "Azure Administrator" means knowing where things live, how to create and connect them safely, and how to keep them running and affordable. Every chapter after this one is really just one category of "thing you can rent" explained in depth, in the same order Microsoft's own exam guide presents it.

0.1 How Azure is organized: the management hierarchy

Before you can administer Azure, you need the map. Everything in Azure lives inside a hierarchy of containers. Understanding this hierarchy is the single most important foundation for the whole exam, because governance, billing, and access all flow through it.

Level	What it is
Management group	A container for multiple subscriptions, so you can apply governance to many at once
Subscription	A billing and access boundary; resources are billed to a subscription
Resource group (RG)	A folder that holds related resources sharing a lifecycle
Resource	An individual service instance — a VM, storage account, database, etc.

ANALOGY

Think of a company's filing system. The **tenant** is the whole company. **Management groups** are divisions. **Subscriptions** are departments with their own budgets. **Resource groups** are project folders. **Resources** are the actual documents. Rules set at a higher level flow down to everything beneath.

MEMORIZE THIS

The hierarchy, top to bottom: **Management group** → **Subscription** → **Resource group** → **Resource**. Governance and access assigned at a higher scope are *inherited* by everything below. A resource lives in exactly one resource group.

EXAM TIP

A resource group is a logical container, not a location boundary — resources in one RG can be in different regions. But the RG itself has a region (where its metadata lives).

0.2 Regions and availability zones

Azure runs in datacenters grouped into **regions** (like East US or West Europe) around the world. Within many regions are **availability zones** — physically separate datacenters with independent power and networking. Spreading resources across zones protects you if one datacenter fails. Some regions are paired with another region for disaster recovery.

MEMORIZE THIS

Region = a geographic set of datacenters. **Availability zone** = a physically separate datacenter within a region (protects against datacenter failure). **Region pairs** support cross-region recovery. Choose regions for latency, compliance/residency, and feature availability.

0.3 Azure Resource Manager and your admin tools

DEFINITION

Azure Resource Manager (ARM): The deployment and management layer of Azure. Every request — from the portal, CLI, PowerShell, or a template — goes through ARM, which handles authentication, applies RBAC, and creates/updates resources.

You'll manage Azure through several interfaces, and the exam tests all of them:

Tool	What it is	Best for
Azure portal	The web GUI (portal.azure.com)	Learning, one-off tasks, visual work
Azure CLI	Cross-platform command line (az commands)	Scripting on Linux/macOS/Windows
Azure PowerShell	PowerShell module (Az cmdlets)	Windows-centric scripting/automation
Cloud Shell	Browser-based shell with CLI/PowerShell built in	Running commands without local setup
ARM templates / Bicep	Declarative infrastructure-as-code files	Repeatable, automated deployments

COMMAND EXAMPLE

```
# Azure CLI: create a resource group
az group create --name myRG --location eastus
# PowerShell equivalent
New-AzResourceGroup -Name myRG -Location eastus
```

IN PLAIN ENGLISH

Whatever tool you use, it all funnels through Resource Manager. Learn the portal to understand concepts, then learn the CLI/PowerShell verbs because the exam will show you commands and ask what they do.

EXAM TIP

Know the command verbs: Azure CLI uses **az <service> create/list/show/delete**; PowerShell uses **Verb-AzNoun** (New-, Get-, Set-, Remove-). If you see **az**, it's CLI; if you see **-Az**, it's PowerShell.

0.4 About the AZ-104 exam

Fact	Detail
Full name	Microsoft Certified: Azure Administrator Associate
Level	Associate (Microsoft recommends ~6 months hands-on Azure)
Format	40–60 questions: multiple choice, drag-and-drop, case studies, and interactive labs
Time	100 minutes
Passing score	700 / 1000 (scaled)
Cost	~\$165 USD (varies by region)
Validity	1 year; renew free online via Microsoft Learn
Latest update	17 April 2026 (Entra naming, Container Apps, Bicep, deployment focus)

The five skill areas, their official sub-groups, and how this book maps to them — in the same order, start to finish:

Part	Skill area	Weight	Chapters
1	Manage Azure identities and governance	20–25%	1–3
2	Implement and manage storage	15–20%	4–6
3	Deploy and manage Azure compute resources	20–25%	7–10
4	Implement and manage virtual networking	15–20%	11–13
5	Monitor and maintain Azure resources	10–15%	14–15

EXAM TIP

Identities/governance and compute are the two heaviest areas (up to 25% each) — together nearly half the exam. But networking and storage are dense and detail-heavy, and the interactive labs often come from compute and networking. Don't skip anything.

0.5 How AZ-104 questions actually get tricky

Having now seen real exam material, a pattern is worth naming up front: AZ-104 rarely asks “what does X do?” in isolation. It asks you to pick between two or three plausible-sounding options that all sound like they could work, where only one satisfies every constraint in the scenario. The details that decide the answer are almost always **exact numbers** (SLA percentages, priority ranges, retention days, IP counts) or a **precise default behavior** (what happens if you don't configure something). Every chapter from here on adds a “Sharpen the Edge” section with exactly these numbers and defaults.

MEMORIZE THIS

When two answers both sound technically correct, look for the **most specific constraint in the question** — a region, a cost requirement, an exact number, or a phrase like “without downtime” or “without opening inbound ports.” That constraint is almost always what eliminates the wrong-but-plausible option.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. List the Azure management hierarchy from top to bottom.

A. Management group → Subscription → Resource group → Resource.

Q2. What does Azure Resource Manager do?

A. It's the management layer every request passes through — handling auth, RBAC, and resource deployment.

Q3. What's the difference between a region and an availability zone?

A. A region is a geographic set of datacenters; an availability zone is a physically separate datacenter within a region.

Q4. You see the command 'New-AzVM'. Is that CLI or PowerShell?

A. PowerShell (Verb-AzNoun). Azure CLI would be 'az vm create'.

CHAPTER SUMMARY

KEY TAKEAWAYS

- Azure hierarchy: management group → subscription → resource group → resource; higher scopes are inherited.
- Regions group datacenters; availability zones are separate datacenters within a region; region pairs aid recovery.
- Everything flows through Azure Resource Manager; manage via portal, CLI (az), PowerShell (Az), Cloud Shell, or templates/Bicep.
- AZ-104 is a 100-minute associate exam with interactive labs; pass at 700/1000. This book follows the official study guide's exact order throughout.

PART F

Foundations

Before the Objectives: Foundations

Everything from here through the next five chapters teaches the same five domains as the rest of this book — identity and governance, storage, compute, networking, monitoring and recovery — but with a different goal. Where the rest of the book (which follows, unchanged, right after this section) teaches you to recognize and correctly configure each individual objective, this section teaches WHY each thing exists, WHEN it genuinely works, WHEN it breaks down, and — after introducing every piece separately — HOW they all combine into one real, working system.

Read this section once, slowly, ideally before or alongside your first pass through the objective-by-objective chapters. It won't teach you a new fact you can quote on exam day; it teaches you the reasoning that lets you answer a question you've never seen worded this way before — which is exactly the skill real AZ-104 candidates say separates a first-attempt pass from a retake.

MEMORIZE THIS

Five Foundations chapters, one per domain, each ending with a “Putting It All Together” synthesis showing a complete, real architecture assembled from every piece taught in that domain's objective chapters.

Foundations: Identity & Governance

WHAT YOU'LL LEARN

- Why identity had to become the control point, not the network
- Why RBAC exists, and the specific problem it does and doesn't solve
- Why Azure Policy exists as a second, separate control system from RBAC
- Why the management hierarchy exists, and how governance actually scales
- How every piece combines into one working governance model

IN PLAIN ENGLISH

Part 1 of this book (Chapters 1–3) taught you HOW to create users, assign roles, and configure policy. This Foundations chapter steps back and answers a different question: why does any of this exist at all, in this particular shape? Understanding the “why” is what lets you answer a question you've never seen before, instead of only recognizing ones you've memorized.

F1.1 Why identity became the control point

For decades, security was mostly about the network. If you were physically in the building, plugged into the office network, you were trusted. Firewalls existed to keep outsiders out; once you were inside the perimeter, you could reach most things. This worked because the assumption held: employees, data, and applications all lived inside a boundary you controlled.

That assumption broke. Employees work from home, coffee shops, and airports. Applications moved to the cloud, outside any network you control. Data lives in a storage account reachable from anywhere on Earth with the right URL. The network perimeter didn't get weaker — it stopped existing as a meaningful boundary at all. Something else had to become the checkpoint every request passes through, and the only thing present in every single request — whether it comes from the office, a home laptop, or an automated script — is an **identity** claiming to be someone or something.

WHY THIS EXISTS

Identity became the security perimeter not because it's a better idea in the abstract, but because it's the only checkpoint that still exists once the network boundary is gone. Every single request, from anywhere, carries an identity. Nothing else is universally present anymore.

WORKS WELL WHEN

Verifying identity works as a control point when you can express real policy around it — who is this, is the device healthy, is the request unusual — and when every resource is willing to check that identity before acting. This is why Azure asks EVERY service, from a storage account to a VM to a subscription, to check RBAC before doing anything.

BREAKS DOWN WHEN

Identity-based control breaks down the moment a credential is stolen and the thief presents it convincingly — identity alone can't tell a legitimate user from an attacker holding valid credentials. This is precisely why MFA, Conditional Access, and risk-based sign-in (SC-300 territory, beyond this exam) exist as a second layer on top of identity, not a replacement for it. AZ-104 assumes identity is already verified and focuses on what that identity is then allowed to do — which is exactly RBAC's job, next.

F1.2 Why RBAC exists — and the one problem it deliberately does NOT solve

Once you know who someone is, you still need to decide what they can do. The naive approach — granting permissions to each person individually, resource by resource — falls apart almost immediately at real scale. A company with 200 employees and 500 resources doesn't have 700 things to manage; it has up to 100,000 individual person-to-resource decisions if handled one at a time. RBAC exists specifically to collapse that explosion: define a **role** once (a bundle of permissions), and assign people to it. The math changes from “people times resources” to “people times roles,” which is dramatically smaller and, critically, stays understandable as the organization grows.

WORKS WELL WHEN

RBAC works extremely well when access needs are **relatively stable and role-shaped** — “anyone on the DBA team needs Contributor on these three resource groups, indefinitely, until they change teams.” Most day-to-day administration fits this pattern.

BREAKS DOWN WHEN

RBAC breaks down for access that is **temporary, one-off, or needs a reason recorded** — “Jo needs Contributor on this one resource group for the next four hours to fix an incident, and we need a record of why.” A standing RBAC assignment can technically do this, but it leaves permanent access sitting around after the need has passed, which is exactly the kind of privilege creep that makes environments harder to audit over time. (The purpose-built answer to this gap, Privileged Identity Management, is SC-300 material — AZ-104 wants you to recognize RBAC's shape and its limits, not necessarily fix them.)

EXAM TIP

When a scenario describes access that should expire, requires approval, or needs justification recorded, and the answer options only offer RBAC role assignments, the correct read is usually “this is the closest AZ-104-scope answer, even though it's an imperfect fit” — unless PIM is explicitly offered as an option, since PIM sits just outside this exam's core scope but occasionally appears as a distractor or a correct answer in identity-heavy questions.

F1.3 Why Azure Policy exists as a SEPARATE system from RBAC

Here's a scenario RBAC cannot solve on its own: you've correctly given a developer Contributor on a resource group so they can do their job. Contributor lets them create anything — including, say, a storage account with public access wide open, or a VM in a region your company isn't allowed to use for compliance reasons. RBAC answered “who can act,” correctly. It has no vocabulary at all for “but what are they allowed to create.” That gap is the entire reason Azure Policy exists as a distinct system, not a feature bolted onto RBAC.

ANALOGY

RBAC is the key that opens the office door. Azure Policy is the building code that says what you're allowed to build once you're inside — you can legitimately hold the key and still not be allowed to install something that violates code. Two independent systems, two independent questions.

WORKS WELL WHEN

Policy works well for rules you can state as a clear, mechanical condition: allowed regions, required tags, allowed SKUs, mandatory encryption. Anything you can phrase as “this property must/must not equal X” is Policy's natural territory.

BREAKS DOWN WHEN

Policy breaks down for judgment calls that need human context: “is this particular exception justified?” Policy can Deny or Audit against a rule, but it can't weigh circumstances — which is why real organizations pair broad Policy enforcement with an exemption/exception process for the legitimate edge cases, rather than trying to encode every possible exception into the policy definition itself. An overly rigid Deny policy with no exception path is a common real-world operational mistake, not just an exam trap.

F1.4 Why the management hierarchy exists

Policy and RBAC both need a scope to apply at. If every policy and every role had to be reassigned individually on every subscription as a company grows from 3 subscriptions to 300, governance would collapse under its own weight exactly the way person-by-person permissions did. Management groups exist to solve the SAME scaling problem RBAC solved for people, applied one level up: instead of repeating a policy assignment 300 times, assign it once, at a management group, and let inheritance carry it down.

WHY THIS EXISTS

Every layer of the Azure hierarchy — management group, subscription, resource group — exists to answer the same underlying question at a different scale: “how do I apply one decision to many things without repeating myself, and without losing the ability to make an exception somewhere specific?”

F1.5 Putting it all together: one governance model, assembled

You've now met identity, RBAC, Azure Policy, locks, tags, and the management hierarchy as separate pieces across Chapters 1–3. Here is how a real organization assembles them into one coherent system — read this as the answer to “how does this actually work together,” not as new material to memorize.

- **At the top**, a management group holds every subscription. An Azure Policy initiative is assigned ONCE, here, requiring approved regions and mandatory tags — it inherits down to every subscription and resource group without being reconfigured anywhere else.
- **At the subscription level**, budgets and cost alerts watch spend for that business unit, and broad RBAC roles (like a subscription-wide Reader for a finance team that needs visibility but not control) are assigned here, once, instead of on every resource group inside it.
- **At the resource group level**, narrower RBAC assignments give specific teams Contributor on just their own workload's resources — inheriting the Policy and budget controls from above automatically, with no extra configuration.
- **On individual critical resources**, a CanNotDelete lock adds a final backstop that not even an Owner can bypass without deliberately removing the lock first — protecting against accidents, not malice, since anyone with access could still remove the lock if they truly intended to.
- **Underneath all of it**, every single check — the Policy evaluation, the RBAC check, the lock check — happens because a request arrived carrying an identity, which is the one constant thread connecting every layer back to Foundations F1.1.

MEMORIZE THIS

Notice what each layer does NOT do: RBAC never decides what's allowed to exist (that's Policy). Policy never decides who can act (that's RBAC). Locks never decide who has access (they're a blunt yes/no on an action, layered on top of both). Each piece has exactly one job, and the system works because none of them try to do each other's job.

EXAM TIP

When a governance scenario question feels complicated, identify which of the three questions it's actually asking — *who can act* (RBAC), *what's allowed to exist* (Policy), or *can this specific action happen at all right now* (a lock) — and the right service usually falls out immediately once you've sorted the question into the right bucket.

Foundations: Storage

WHAT YOU'LL LEARN

- Why storage tiers exist as an economic tradeoff, not a technical one
- Why redundancy comes in four flavors instead of one 'safe' option
- Why THREE different access methods exist for the same data
- Why blob protection is three separate features, not one
- How it all combines into one storage design

IN PLAIN ENGLISH

Chapters 4–6 taught you how to configure storage access, accounts, and blob features. This chapter explains why storage has so many knobs in the first place — each one exists because it trades off against something else, and knowing the trade lets you pick correctly even in a scenario you haven't seen worded this way before.

F2.1 Why access tiers exist: a physical and economic tradeoff, not a preference

Storing data “ready to serve instantly” costs more than storing it “somewhere, eventually retrievable.” This isn't a Microsoft pricing decision so much as a physical reality reflected in pricing: keeping data on fast, always-spinning, immediately-available media costs more than keeping it on cheaper media that takes time to bring back online. Tiers exist so YOU choose where each piece of data sits on that spectrum, instead of paying premium prices for data nobody's touched in two years.

WORKS WELL WHEN

Tiering works when access patterns are predictable — you genuinely know this data gets hit constantly (Hot) or essentially never after 90 days (Archive). Lifecycle management rules exist specifically to automate this instead of asking you to guess and manually move things.

BREAKS DOWN WHEN

Tiering fails you when access patterns are unpredictable — if “rarely accessed” data suddenly needs to be read in bulk (a legal discovery request touching two years of Archive-tier records, say), you pay both the retrieval cost AND the rehydration delay all at once, for everything, right when you can least afford to wait. This is precisely why the minimum-stay penalties exist too — they discourage guessing wrong about a tier and moving data back out early.

F2.2 Why redundancy comes in four flavors instead of one 'safe' option

It would be simpler if Azure just picked “the safe option” for everyone. It doesn't, because “safe enough” depends on what you're protecting against, and every extra layer of protection costs more. LRS protects against a disk failing. ZRS protects against a whole datacenter failing. GRS protects against a whole region failing. Each step outward protects against a bigger, rarer disaster — and costs more, because Azure is now keeping physically separate copies further and further apart.

WHY THIS EXISTS

The redundancy ladder exists because “how much disaster can you survive” and “how much are you willing to pay” are two independent dials, and different data genuinely needs different answers. A dev/test storage account and a company's financial records should not cost the same to protect, because they don't need the same protection.

BREAKS DOWN WHEN

Redundancy alone fails to protect against the failure mode people most often actually hit: a human deleting or overwriting the wrong thing on purpose or by mistake. Every redundancy tier faithfully replicates a mistake to every copy just as fast as it replicates good data — which is exactly why soft delete and versioning exist as an entirely separate protection layer (F2.4), not a feature of redundancy at all.

F2.3 Why three different access methods exist for the same data

Access keys, SAS tokens, and identity-based access all answer “how do you prove you're allowed to touch this data,” but they exist because they solve genuinely different trust problems, not because Microsoft couldn't settle on one.

Method	The trust problem it solves
Access keys	An internal system needs full, ongoing access and you control that system completely
SAS tokens	An external party needs LIMITED, TEMPORARY access, and you don't want to give them a real identity in your directory at all
Identity-based (Entra + RBAC)	An internal user or app already has an identity you manage, and you want ONE consistent place (RBAC) to control everything they can touch, storage included

WORKS WELL WHEN

Each method works well exactly inside the trust problem it was built for — keys for fully-trusted internal automation, SAS for scoped external/temporary sharing, identity-based for anything already inside your Entra directory.

BREAKS DOWN WHEN

Using the wrong one for the situation is where real incidents happen: a SAS token used for long-term internal system access becomes a silent, hard-to-rotate secret buried in a config file somewhere. An access key handed to an external partner “just this once” becomes a permanent, full-account secret sitting outside your control forever. The method isn't broken — it's being used to solve a problem it wasn't designed for.

F2.4 Why blob protection is three separate features, not one

Soft delete, versioning, and snapshots all sound like “back up my data,” but they each answer a different accident.

- **Soft delete** exists because the most common accident is *deleting* the wrong thing.
- **Versioning** exists because the second most common accident is *overwriting* the right thing with the wrong content.
- **Snapshots** exist because sometimes you want a deliberate, known-good checkpoint BEFORE a risky change — not an automatic safety net, but a point you chose on purpose.

WHY THIS EXISTS

Three features instead of one exist because “protect my data” is actually three different failure modes (deleted, overwritten, or “I want to go back to a specific known point”), and a single generic “backup” switch would have to guess which one you meant.

F2.5 Putting it all together: one storage design, assembled

Here's how a real storage design for, say, a document-management system combines everything from Chapters 4–6 into one coherent decision, piece by piece:

- **Redundancy choice** comes first, driven by business risk tolerance: GRS, because losing customer documents in a regional outage is unacceptable, but the extra cost of GZRS/RA-GZRS isn't justified for this workload.
- **Access method** comes next: internal employees authenticate via Entra ID + RBAC (no shared secrets to manage or rotate); an external auditor who needs read access to one folder for two weeks gets a scoped, time-limited SAS token instead of a guest account.
- **Network access** layers on top: a private endpoint so the traffic never crosses the public internet, with the storage firewall's default action set to Deny.
- **Tiering** is decided by access pattern: active case files stay Hot; closed cases move to Cool after 30 days via a lifecycle rule; anything closed over 2 years old moves to Archive.
- **Protection** stacks on top of all of it: soft delete and versioning are on by default across every tier, so an accidental delete or overwrite is recoverable regardless of which tier the file currently sits in.

MEMORIZE THIS

Every one of these five decisions is INDEPENDENT of the others — you choose redundancy, access method, network exposure, tiering, and protection separately, and they combine without conflicting. That independence is exactly why storage has so many settings: each one is a separate dial for a separate risk, not five settings that do the same thing five different ways.

EXAM TIP

A storage scenario question is almost always testing whether you can identify WHICH of these five independent dials the requirement is actually about. “Minimize cost for rarely-accessed data” is tiering. “Protect against a regional outage” is redundancy. “No public internet exposure” is networking. Don't reach for a redundancy answer when the question is actually about access method, and vice versa.

Foundations: Compute

WHAT YOU'LL LEARN

- Why infrastructure as code exists, and what it costs to adopt
- Why VMs, containers, and App Service are three points on one spectrum, not three unrelated tools
- Why availability sets AND zones both exist instead of just the stronger one
- How it all combines into one compute design

IN PLAIN ENGLISH

Chapters 7–10 taught you how to deploy VMs, containers, and App Service, and how to do it with Bicep instead of clicking. This chapter is about why you have three fundamentally different ways to run the same application, and why that's a real, meaningful choice rather than a matter of taste.

F3.1 Why infrastructure as code exists, and what it actually costs

Clicking through the portal to build one environment is fine. The problem shows up the second time: rebuilding the same environment for staging, or disaster recovery, or a second region, by memory, by hand, is where subtle differences creep in — a setting you forgot, a name you typed slightly differently. Those small drifts are invisible until something breaks in production that worked fine in staging, for a reason nobody can find because nobody wrote down what was actually different.

WHY THIS EXISTS

IaC exists to make environments reproducible **ON PURPOSE**, not by careful memory. The template becomes the single source of truth, so “what does production actually look like” has one answer instead of “whatever we remember clicking.”

WORKS WELL WHEN

IaC pays off the moment you need to build something more than once — a second environment, a disaster-recovery region, or just the confidence to tear down and rebuild without fear.

BREAKS DOWN WHEN

IaC has a real, honest cost: writing and testing a template takes longer than clicking the same thing once. For a genuine one-off resource you'll never rebuild, portal clicking is not “the wrong answer” — it's a reasonable choice the exam generally doesn't test directly, but worth knowing in the real world so you don't over-engineer a single throwaway resource.

F3.2 Why VMs, containers, and App Service are one spectrum, not three unrelated tools

These three aren't really separate categories — they're three points along a single spectrum trading control for convenience.

	VM	Container (ACI/Container Apps)	App Service
You manage	OS, runtime, app, patching, everything	Just the app + its dependencies, packaged	Just your code
Azure manages	Physical hardware only	The OS and orchestration underneath	Everything except your code

	VM	Container (ACI/Container Apps)	App Service
Control	Maximum	Middle ground	Minimum, but zero ops burden

WHY THIS EXISTS

This spectrum exists because “how much do you want to manage yourself, versus hand to Azure” is a real, continuous tradeoff, not a binary choice — different applications, teams, and legacy constraints sit at genuinely different points on it.

WORKS WELL WHEN

VMs work when you need OS-level control — a legacy app that needs a specific Windows version, custom drivers, or software that assumes it owns the whole machine. Containers work when your app is already packaged that way and you want portability without owning OS patching. App Service works when you just have code and want the fastest path from code to a running, patched, scalable web endpoint.

BREAKS DOWN WHEN

Each choice fails outside its zone: a VM for a simple stateless web app is needless operational burden (you're now patching an OS nobody asked you to own). App Service fails when your app genuinely needs OS-level access or an unsupported runtime. Containers fail when your app was never built to be stateless/portable and assumes a persistent, specific machine underneath it.

F3.3 Why availability sets AND zones both exist

If zones protect against a bigger disaster (a whole datacenter) than sets (hardware within one datacenter), why would anyone choose the weaker option? Because zones aren't available everywhere, and even where they are, not every VM size supports them — and because zones introduce cross-datacenter network latency that sets don't have, since zone-spread VMs are physically further apart.

WORKS WELL WHEN

Zones work when your region supports them, your VM size supports them, and the small latency cost of physically separated datacenters is acceptable for your workload — which it is for most web-tier and app-tier workloads.

BREAKS DOWN WHEN

Zones fail as an option entirely in regions or for VM sizes that don't support them — at which point an availability set is not a downgrade, it's the only zone-style protection actually available. Zones can also be the wrong choice for latency-sensitive workloads (like some clustered database configurations) where the physical distance between zones introduces delay a same-datacenter availability set wouldn't have.

F3.4 Putting it all together: one compute design, assembled

Here's how a real production web application combines everything from Chapters 7–10 into one coherent deployment:

- **The whole thing is defined in Bicep**, checked into source control, so staging and production are provably identical and a disaster-recovery rebuild is a redeploy, not a memory exercise.
- **The compute choice** is App Service, because this is a standard web app with no OS-level requirement — maximum convenience, matching the spectrum decision from F3.2.
- **The App Service plan is Standard tier**, specifically because the team needs deployment slots for zero-downtime releases and autoscale for traffic spikes — both of which require this tier, not a lower one.
- **Deployment goes through a staging slot first**, tested, then swapped — the release process the plan tier was chosen to enable.

- **If the app later needs OS-level access** for a specific legacy dependency, that ONE component moves to a VM in an availability set within the same architecture, without needing to redesign everything else — because the compute spectrum (F3.2) means you can mix points on it within one system.

MEMORIZE THIS

The compute decision and the availability decision are SEPARATE choices that combine, not one combined choice — you pick WHERE on the VM/container/App Service spectrum you sit, and separately (for VM-based choices) HOW you protect against failure. A scenario question is often really asking about just one of these two axes at a time.

Foundations: Networking

WHAT YOU'LL LEARN

- Why networks are segmented instead of left flat
- Why NSGs exist as a specific, limited tool — and what they deliberately don't do
- Why remote access tooling (Bastion) exists as a dedicated service instead of just a firewall rule
- Why hub-and-spoke became the standard pattern
- How it all combines into one network design

IN PLAIN ENGLISH

Chapters 11–13 taught you how to build and secure a network piece by piece — VNets, NSGs, Bastion, endpoints, load balancers. This chapter explains why networking looks the way it does: every piece exists as a response to a specific failure mode of the simpler design that came before it.

F4.1 Why networks are segmented instead of left flat

A single flat network, where every resource can reach every other resource, is the simplest possible design — and the worst one from a security standpoint. If an attacker (or a misconfigured app) compromises ANY one resource, they can reach everything else with no additional effort. Segmentation exists to change that math: a compromise of one subnet shouldn't automatically mean a compromise of every subnet.

WHY THIS EXISTS

Subnets and NSGs exist to limit **blast radius** — the idea that a failure or compromise in one place should cost you as little as possible elsewhere, not that failures never happen at all.

WORKS WELL WHEN

Segmentation works when you draw boundaries around genuinely different trust levels — a database tier that should only ever talk to an app tier, an app tier that should only talk to a specific web tier, is a meaningful, defensible boundary.

BREAKS DOWN WHEN

Segmentation fails to help — and just adds friction — when boundaries are drawn arbitrarily rather than around real trust differences. Splitting resources across subnets that all trust each other completely and all need to talk to each other freely doesn't reduce blast radius; it just adds NSG rules to maintain for no real security gain.

F4.2 Why NSGs exist as a LIMITED tool, on purpose

An NSG only looks at source, destination, port, and protocol — it has no idea what's actually inside the traffic. It can allow port 443 without knowing or caring whether the HTTPS request inside is a legitimate API call or an attack payload. This isn't an oversight; NSGs are deliberately simple and fast because they run on every single network interface in Azure, constantly. A tool that inspected every byte of every packet at that scale would be far more expensive and slower.

WORKS WELL WHEN

NSGs work perfectly for the job they're built for: coarse, fast, “should this kind of traffic even be allowed here” filtering, at massive scale, with essentially no performance cost.

BREAKS DOWN WHEN

NSGs fail the moment a question needs actual content inspection — blocking SQL injection, inspecting HTTP payloads, detecting malware in a file upload. That's not a gap in NSG configuration; it's outside what an NSG was ever designed to do. Azure Firewall and a Web Application Firewall exist specifically to fill that gap (they're not in AZ-104's core objectives, but recognizing the boundary of what an NSG can do is exactly what separates a real understanding from having memorized the rule table).

F4.3 Why Bastion exists as a dedicated service, not just an NSG rule

You could, in theory, just write an NSG rule allowing RDP/SSH from your home IP and call it solved. Two real problems with that: your home IP changes, so the rule needs constant updating or you widen it dangerously; and the port is still technically open to the internet for anyone who guesses or scans for it, protected only by a rule you have to remember to keep current. Bastion exists to remove the open port entirely — there's no RDP/SSH port facing the internet AT ALL, on any IP, ever, because the connection happens through the Azure portal and Bastion's own infrastructure instead.

WHY THIS EXISTS

Bastion exists because the safest open port is no open port. It doesn't make RDP/SSH access to the internet more tightly controlled — it removes that access surface from existing at all.

F4.4 Why hub-and-spoke became the standard pattern

Once an organization has more than one or two VNets, a new problem appears: shared services (a firewall, Bastion, VPN gateway) either need to be duplicated in every VNet — expensive and inconsistent — or centralized somewhere every other VNet can reach. Hub-and-spoke is that centralization: one hub VNet holds the shared services once, spoke VNets hold workloads and peer back to the hub to reach those shared services, and spokes don't peer directly to each other (matching the non-transitive nature of peering you learned in Chapter 11).

WORKS WELL WHEN

Hub-and-spoke works well as an organization grows past a handful of VNets and needs consistent, centrally-managed shared services without duplicating them everywhere.

BREAKS DOWN WHEN

For a single, simple workload with one VNet, hub-and-spoke is unnecessary complexity — the pattern earns its cost at scale, not on day one with a single small application.

F4.5 Putting it all together: one network design, assembled

Here's how a real secure network combines everything from Chapters 11–13 into one working system:

- **A hub VNet** holds Azure Bastion (F4.3) in its own dedicated subnet — the single, centralized point of administrative access for everything downstream.
- **A spoke VNet**, peered to the hub, holds the actual workload: a web-tier subnet and a data-tier subnet, segmented (F4.1) so a compromise of the web tier doesn't automatically expose the database.
- **NSGs** (F4.2) on each subnet enforce that segmentation mechanically: the data subnet only accepts traffic from the web subnet, never from the internet directly, and admin access only arrives from the hub.
- **A private endpoint** lets the app reach its storage account without that traffic ever touching the public internet, extending the same “no unnecessary exposure” principle Bastion applied to VM access.

- **A Standard SKU load balancer** in front of the web tier distributes traffic across multiple instances — and because Standard is secure-by-default, its own NSG rules had to be deliberately opened for AzureLoadBalancer, which is the segmentation principle applying even to Azure's own internal traffic.

MEMORIZE THIS

Every piece in this design answers the same question at a different layer: “what's the smallest amount of access that still lets this work?” VNets/subnets answer it at the network layer, NSGs at the traffic-rule layer, Bastion at the admin-access layer, private endpoints at the PaaS-access layer. Recognizing that repeated pattern is what lets you answer a networking scenario you've never seen phrased this way before.

Foundations: Monitoring & Recovery

WHAT YOU'LL LEARN

- Why metrics and logs both exist instead of just one data type
- Why alerting exists, and what it deliberately doesn't guarantee
- Why backup and Site Recovery are two separate services, not one
- How it all combines into one operational readiness model

IN PLAIN ENGLISH

Chapters 14–15 taught you how to configure monitoring, alerts, backup, and Site Recovery. This final Foundations chapter is about why observability and recovery need this many distinct pieces — and how they form one continuous story: know something's wrong, know what happened, and be able to recover either way.

F5.1 Why metrics and logs both exist instead of just one

You might reasonably ask: why not just have one kind of monitoring data? Because “fast” and “detailed” pull in opposite directions. A number (CPU is at 94%) can be sampled, transmitted, and alerted on in near real time because it's tiny. A detailed event record (exactly which process did what, at what time, with what parameters) carries far more information but costs more to store, transmit, and query — which is why it lives in a queryable workspace instead of a lightweight time-series store.

WHY THIS EXISTS

Metrics and logs exist as two separate systems because “alert me fast” and “let me investigate deeply after the fact” are different jobs with different cost profiles, and trying to do both with one data type would compromise both.

WORKS WELL WHEN

Metrics work well for “is something wrong right now” — fast thresholds on numeric values. Logs work well for “what exactly happened, and why” after you already know something needs investigating.

BREAKS DOWN WHEN

Metrics fail you the moment the question is “why” rather than “what” — a CPU spike metric tells you it happened, never what process caused it. Logs fail as a FIRST alert mechanism, because querying a workspace fast enough to catch a live incident as it happens isn't what they're optimized for — which is exactly why alert rules are usually metric-based for speed, with logs as the follow-up investigation tool.

F5.2 Why alerting exists, and what it deliberately doesn't guarantee

Metrics and logs are useless if a human has to remember to go look at them. Alerting exists to flip that around: instead of you watching a dashboard, the system watches itself and tells you when a condition is met. But an alert rule only fires on conditions you thought to define in advance — it cannot warn you about a failure mode nobody anticipated.

BREAKS DOWN WHEN

Alerting fails silently in a specific, common way: an alert rule with NO action group attached, or a notification going to an inbox nobody monitors, fires perfectly and helps no one. This is precisely why Chapter 15's backup reports and alerts matter as much as the backup policy itself — a backup job that's been silently failing for weeks is functionally the same as never having configured backup at all, discovered at the worst possible moment.

EXAM TIP

Whenever a scenario mentions something going unnoticed for a period of time, the gap is almost always a missing or misconfigured alert/action group — not a missing monitoring capability. The data was very likely already being collected; nobody was told about it.

F5.3 Why backup and Site Recovery are two separate services

These solve genuinely different disasters, which is why merging them into one service would actually make both worse at their job.

	Azure Backup	Azure Site Recovery
The disaster it answers	Something inside the system went wrong (deleted, corrupted, encrypted by ransomware)	The system's LOCATION went wrong (the whole region is unavailable)
What you get back	A point-in-time copy of data, restored where you choose	A running, working copy of the whole system, already live elsewhere
Speed characteristic	Optimized for infrequent, deep, space-efficient snapshots	Optimized for continuous, near-real-time replication, ready to activate instantly

WHY THIS EXISTS

Backup is optimized to answer “give me yesterday's version back,” efficiently, for data that changes daily. Site Recovery is optimized to answer “keep a live, current copy running somewhere else, ready to take over in minutes.” Building one service trying to do both would either make backups wastefully continuous or make failover unacceptably stale — splitting them lets each be genuinely good at its one job.

F5.4 Putting it all together: one operational readiness model, assembled

Here's how a real production system combines everything from Chapters 14–15 into one continuous story, from “everything's fine” to “we just recovered from a disaster”:

- **Metrics flow constantly** and a CPU/availability alert rule, linked to a real action group someone actually monitors, catches trouble within minutes.
- **Diagnostic settings route detailed logs** to a Log Analytics workspace, so once an alert fires, an analyst can immediately query KQL to understand exactly what happened — the fast signal (metrics) handed off cleanly to the deep signal (logs).
- **Monitor Insights** gives a pre-built dashboard view of the same underlying data, so day-to-day health checks don't require hand-writing a KQL query every time.
- **If the incident turns out to be data corruption**, Azure Backup restores a clean recovery point — the “something inside went wrong” answer.
- **If the incident turns out to be a full regional outage instead**, Azure Site Recovery fails over to the replicated copy already running in the secondary region — the “the location went wrong” answer, a completely different response to a completely different problem.
- **Backup reports and alerts** run continuously underneath all of this, silently, so the recovery point you need during a real incident is guaranteed to actually exist when you reach for it.

MEMORIZE THIS

The whole operational model is really one repeating idea at every stage: fast, lightweight signals (metrics, backup alerts) catch problems early; deep, detailed systems (logs, recovery points, replicated regions) let you actually fix them. Every piece in Chapters 14–15 is one of those two roles — knowing which role a service plays tells you when to reach for it.

EXAM TIP

This closes the loop back to Foundations F1—F4: identity controls who can act, governance controls what's allowed, compute and storage and networking build the thing, and monitoring/recovery is how you know it's working and get it back when it isn't. Five domains, one continuous system — not five unrelated exam topics.

EXAM REVIEW

Objective by Objective, in the Official Study Guide's Own Order

Everything from here forward is the complete reference edition of this book: every AZ-104 objective, in Microsoft's own official order, with the precise numbers, exam traps, hands-on labs, troubleshooting guide, and case studies you've seen built up across earlier editions.

Use the Foundations section you just read to understand WHY. Use everything from here to the end of the book as your objective-by-objective study and your final review in the days before the exam.

PART 1

Identities & Governance

Microsoft Entra Users & Groups

WHAT YOU'LL LEARN

- Create users and groups
- Manage user and group properties
- Manage licenses in Microsoft Entra ID
- Manage external users
- Configure self-service password reset (SSPR)

IN PLAIN ENGLISH

Zoom out for a second: nothing in Azure can happen without someone (or something) signing in first. This chapter follows Microsoft's own official order exactly: first you create the accounts, then you manage their properties, license them, handle external collaborators, and finally give them a way to reset their own password.

1.1 Creating users and groups

Every person who signs in to your Azure/Microsoft 365 environment needs an identity in Microsoft Entra ID. There are two broad kinds. A **member** is a normal account that belongs to your organization (your employees). A **guest** is an external person invited to collaborate — covered in depth in 1.4. You can create users individually, in bulk via CSV, or by syncing on-prem AD.

COMMAND EXAMPLE

```
# Create a user with Azure CLI
az ad user create --display-name Jo --user-principal-name jo@contoso.com \
  --password P@ssw0rd!
# Bulk-create users in the portal via CSV import
```

Managing access one person at a time doesn't scale, so you also put users into **groups** and grant access to the group instead. Two choices define a group:

Choice	Options	Meaning
Group type	Security	Grants access to resources and roles
	Microsoft 365	Adds shared mailbox/SharePoint for collaboration
Membership	Assigned	You add/remove members manually
	Dynamic	Members are added automatically by a rule on their attributes

ANALOGY

A dynamic group is like a smart playlist: you define a rule ("all songs tagged jazz") and the list fills itself. Change a song's tag and it joins or leaves automatically — just like a user moving departments.

COMMAND EXAMPLE

```
# Example dynamic membership rule (users in Sales)
user.department -eq "Sales"
```

MEMORIZE THIS

Security groups grant access; **Microsoft 365** groups add collaboration. **Dynamic** membership uses attribute rules and requires Entra ID **P1**. Dynamic *user* and dynamic *device* groups are separate — one group can't contain both.

Microsoft Entra Domain Services — a related but distinct service

DEFINITION

Microsoft Entra Domain Services: A managed service that provides traditional domain-join, LDAP, Kerberos/NTLM, and Group Policy capabilities in Azure — without you having to deploy and patch your own domain controller VMs.

It's easy to confuse this with Entra ID itself or with hybrid Entra Connect. Entra Domain Services is for legacy apps and devices that need classic domain-join/LDAP/Kerberos, and it creates its own separate, managed domain — it is NOT the same directory as your on-prem AD or a way to extend on-prem AD into Azure.

MEMORIZE THIS

Entra ID = the modern cloud directory (this whole chapter). **Entra Connect** = syncs on-prem AD to Entra ID (hybrid identity). **Entra Domain Services** = a separate managed domain for legacy LDAP/Kerberos/Group Policy needs. Three different things that often get confused on the exam.

1.2 Managing user and group properties

Creating an account is only the start — day to day administration is mostly about editing properties on the users and groups you already have. On a user object you'll routinely update profile fields (job title, department, manager, contact info), enable or disable sign-in (block sign-in without deleting the account), and reset or set a temporary password. On a group you'll update its name, description, owners (who can manage membership), and add or remove members.

COMMAND EXAMPLE

```
# Update a user's job title and department (CLI)
az ad user update --id jo@contoso.com \
  --job-title "Support Analyst" --department "Helpdesk"
# Block a user from signing in without deleting them
az ad user update --id jo@contoso.com --account-enabled false
```

MEMORIZE THIS

Disabling sign-in (account-enabled=false) blocks access while preserving the account, mailbox, and group memberships — the standard first step for an employee on leave, versus *deleting* the user which is destructive and used for offboarding. A deleted user is **soft-deleted for 30 days** and restorable in that window.

Group **owners** can manage membership without holding any Entra directory role — the standard way to delegate day-to-day group upkeep to a non-admin.

1.3 Managing licenses in Microsoft Entra ID

Features like Entra ID P1/P2 and Microsoft 365 apps are unlocked by **licenses** assigned to users — directly or, more efficiently, through **group-based licensing** (assign the license to a group and every member inherits it, gaining or losing it automatically as they join or leave).

Detail	Exact behavior
License conflicts	If a user has two licenses granting the same service plan (e.g. Exchange Online from two SKUs), Azure disables the duplicate service plan automatically — it does not block the assignment or throw an error
Group-based licensing errors	If a group license can't apply to a member (e.g. a usage-location mismatch), that ONE user shows a licensing error while the rest of the group is unaffected
Removing a user from a licensed group	Immediately removes the inherited license and its normal grace/retention clock starts — same as a direct unassignment
Usage location	A license cannot be assigned at all to a user with no usage location set — a very common “why won't this license apply” root cause

MEMORIZE THIS

Group-based licensing assigns licenses to a group so members inherit them — the scalable default over per-user assignment. Report on license status and errors from the same group-licensing blade in the admin center.

EXAM TIP

A classic trap scenario: “A user reports Exchange Online isn't working even though two licenses are assigned.” The answer is almost never “assign a third license” — it's checking for a **duplicate/conflicting service plan** that got auto-disabled, or a missing **usage location**.

1.4 Managing external users

A **guest** is an external person invited to collaborate — their account really lives in another directory, and you're just inviting them in via Microsoft Entra **B2B collaboration**. Guests are managed like any other identity: you can assign them to groups, grant access to resources, and eventually remove their access when collaboration ends.

User type	Belongs to	Created by
Member	Your organization's tenant	Created directly, or synced from on-prem AD
Guest (external)	Another directory / email	Invited via B2B collaboration (UPN shows #EXT#)

MEMORIZE THIS

Member = internal account. **Guest** = external collaborator invited via **B2B**; their UPN contains **#EXT#**. External Identities settings in the admin center control who can invite guests and which external domains are allowed or blocked.

1.5 Configuring self-service password reset (SSPR)

SSPR lets users reset their own forgotten password after registering authentication methods, cutting help-desk load. You choose the scope (all users or a group), how many methods are required to reset, and enable **combined registration** so users register MFA and SSPR methods in one experience instead of two separate prompts.

COMMAND EXAMPLE

```
# Enable SSPR for a group (portal path)
# Entra admin center > Password reset > Properties > Selected > add group
```

MEMORIZE THIS

SSPR requires users to register methods first; you choose how many methods are required and which are allowed. SSPR for all users requires Entra ID **P1/P2**. **Combined registration** is the single sign-up flow covering both MFA and SSPR methods.

1.6 Sharpen the edge: bulk operations and dynamic-group timing

Detail	Exact behavior
Bulk CSV operations	The admin center's bulk-create/bulk-invite templates are fixed CSV schemas — you cannot add custom columns; anything beyond the template's fields requires PowerShell/Graph
Dynamic group processing	A dynamic membership rule change is NOT instant — Azure evaluates and updates membership on its own schedule, so a test immediately after a rule change can show stale results

1.7 Hands-on lab: create a dynamic group and license it

This mirrors the kind of interactive lab task the real exam gives you. Do this in a free Azure account or a Microsoft 365 developer tenant.

- **Step 1.** In the Entra admin center, go to *Identity > Users > All users > New user > Create new user*. Create two test users, e.g. jo.test@yourtenant.onmicrosoft.com and sam.test@yourtenant.onmicrosoft.com.
- **Step 2.** Open each user's profile and set *Job info > Department* to **Sales** for both. This is the attribute your dynamic rule will match on.
- **Step 3.** Go to *Identity > Groups > New group*. Choose group type **Security**, give it a name, and set Membership type to **Dynamic User**.
- **Step 4.** Click *Add dynamic query* and enter the rule `(user.department -eq "Sales")`, or use the rule builder: Property = Department, Operator = Equals, Value = Sales. Save the group.
- **Step 5.** Wait a few minutes (dynamic membership evaluation isn't instant — this is the exact behavior from section 1.6), then refresh the group's *Members* tab. Both test users should now appear automatically.
- **Step 6.** If you have a Microsoft 365 or Entra ID P1 trial license available, go to the group's *Licenses* blade and assign a license to the group itself — confirm both members show as licensed without you touching either user individually.
- **Step 7 (cleanup).** Change one test user's Department away from Sales and confirm they drop out of the group on the next evaluation cycle — this proves the membership is genuinely rule-driven, not a one-time snapshot.

EXAM TIP

If a lab question asks you to grant several users the same access based on a shared attribute, dynamic groups + group-based licensing is almost always the intended answer over assigning things one user at a time.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. An invited partner's account shows #EXT# in the UPN. What kind of user is it?

A. A guest (external) user, added via B2B collaboration.

Q2. You want a group whose membership updates automatically when someone joins Sales. What do you use?

A. A dynamic group with a membership rule like `user.department -eq "Sales"` (requires P1).

Q3. What must a user do before they can use SSPR?

A. Register the required authentication methods.

Q4. A user reports a duplicate license conflict error. What actually happens to the duplicate service plan?

A. Azure automatically disables it — it doesn't block the assignment or error out.

Q5. What does disabling a user's sign-in preserve, that deleting the user would not?

A. The account, mailbox, and group memberships — reversible, unlike deletion.

CHAPTER SUMMARY

KEY TAKEAWAYS

- Create users and groups first: security vs M365 group type, assigned vs dynamic membership (P1).
- User/group properties: job title/department, disabling sign-in (reversible) vs deleting (30-day soft-delete), group owners.
- Licenses: group-based licensing scales; duplicate service plans auto-disable; usage location is required.
- External users: guests via B2B, UPN contains #EXT#, governed by External Identities settings.
- SSPR: requires registered methods; combined registration unifies MFA + SSPR registration.

Azure RBAC: Access to Resources

WHAT YOU'LL LEARN

- Manage built-in Azure roles
- Assign roles at different scopes
- Interpret access assignments

IN PLAIN ENGLISH

If Chapter 1 was about WHO exists (users and groups), this chapter is about WHAT they're allowed to touch once they're signed in. RBAC (role-based access control) is simply the system for saying "this person, this level of power, on this specific set of resources." Every permission decision in Azure eventually comes down to this one mechanism.

2.1 How Azure RBAC works

DEFINITION

Azure RBAC: Role-based access control for Azure resources: you grant access by creating a **role assignment**, which binds a **security principal** (user, group, or service principal) to a **role definition** (a set of allowed actions) at a **scope** (the level it applies to).

Every RBAC grant is three things answered together: **who** (the principal), **what they can do** (the role), and **where** (the scope). Miss any one and the grant makes no sense.

ANALOGY

A role assignment is like a building pass. The *person* is the principal, the *pass type* (visitor, staff, manager) is the role, and the *Floors it opens* are the scope.

2.2 Managing built-in Azure roles

Role	What it can do
Owner	Full access to everything, INCLUDING granting access to others
Contributor	Manage everything EXCEPT granting access (no RBAC)
Reader	View only; no changes
User Access Administrator	Manage user access / role assignments only

Beyond these four, Azure ships dozens of narrower, service-specific built-in roles:

Built-in role	Scope of power
Virtual Machine Contributor	Manage VMs, but cannot manage the VNet or storage account they depend on
Storage Account Contributor	Manage storage accounts, but cannot access the data inside via RBAC alone (needs a data-plane role too)
Network Contributor	Manage networking resources (VNets, NSGs, public IPs), not VMs or storage
Backup Contributor	Manage Azure Backup (create vaults, policies, trigger backup/restore)
Security Reader	Read-only access to security posture, no ability to change anything

MEMORIZE THIS

Contributor can do almost anything EXCEPT assign roles. Only **Owner** and **User Access Administrator** can grant access to others. Narrow, service-specific Contributor roles are scoped tightly — **Virtual Machine Contributor** cannot also create a new VNet for that VM.

Custom roles, when no built-in role fits

A custom role definition (in JSON) has four key sections: **Actions** (allowed management operations, using wildcards like `Microsoft.Compute/virtualMachines/*`), **NotActions** (exclusions carved out of Actions — NOT a deny, just a subtraction), **DataActions/NotDataActions** (the same idea for data-plane operations), and **AssignableScopes** (which subscriptions/RGs the role can even be assigned at).

MEMORIZE THIS

NotActions is not a Deny. If a user has a second role granting a NotActions-excluded permission through a different assignment, they still get it — NotActions only carves permissions out of that specific role definition.

2.3 Assigning roles at different scopes

You can assign a role at four scope levels, matching the management hierarchy from Chapter 0. Crucially, a role assigned at a higher scope is **inherited** by everything beneath it.

Scope (broadest first)	Assigning here grants access to...
Management group	All subscriptions and resources under it
Subscription	All resource groups and resources in it
Resource group	All resources in that group
Resource	Just that single resource

COMMAND EXAMPLE

```
# Assign Reader at a resource group scope (CLI)
az role assignment create --assignee jo@contoso.com \
  --role Reader --resource-group myRG
```

EXAM TIP

RBAC is **additive** — your effective access is the sum of all your assignments. But a **deny assignment** overrides allow. Assign at the narrowest scope that does the job (least privilege). Reader at a subscription means Reader on everything inside it.

2.4 Interpreting access assignments

Because roles inherit and stack, a user's true access is often the sum of several assignments at several scopes, not just the one you're looking at. Interpreting access correctly means checking every scope that applies to that principal — the portal's *Access control (IAM) > Check access* view does this for you, or use the CLI.

COMMAND EXAMPLE

```
# List every role assignment a user has, at any scope, in a subscription
az role assignment list --assignee jo@contoso.com --all
```

MEMORIZE THIS

When asked “what can this user actually do,” remember RBAC power is **additive across every assignment and every scope** — a Reader at the subscription plus a Contributor at one resource group means Contributor there and Reader everywhere else, not a conflict to resolve.

2.5 Sharpen the edge: RBAC vs Entra roles

Two different role systems exist, and they're easy to confuse. **Azure RBAC** controls access to Azure *resources* (VMs, storage). **Entra roles** control administration of the *directory* (users, groups, Entra settings) — the roles you actually used back in Chapter 1.

MEMORIZE THIS

Azure RBAC = resources (Owner/Contributor/Reader at mgmt group/sub/RG/resource). **Entra roles** = directory objects (Global Administrator, User Administrator). Global Administrator is an Entra role, not an Azure RBAC role — the two systems don't share role definitions.

2.6 Hands-on lab: assign a scoped role and verify inheritance

- **Step 1.** In the Azure portal, navigate to a resource group you own and open *Access control (IAM) > Add > Add role assignment*.
- **Step 2.** Select the **Reader** role, then under *Members* pick one of the test users from Chapter 1's lab. Review + assign.
- **Step 3.** Open a resource *inside* that resource group (e.g. a storage account) and go to its own *Access control (IAM) > Role assignments* tab. You won't see the assignment listed here directly — click *View my access* or *Check access* and search the test user to confirm Reader shows as **inherited** from the parent resource group.
- **Step 4.** Now assign that same user **Contributor** directly on just that one storage account.
- **Step 5.** Run `az role assignment list --assignee <user> --all` from Cloud Shell and confirm you see BOTH assignments listed — Reader at the resource group scope and Contributor at the resource scope — proving RBAC is additive, not a single resolved value.
- **Step 6 (cleanup).** Remove both role assignments when done to keep your subscription tidy.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. Which built-in role can manage all resources but not grant access to others?

A. Contributor.

Q2. You assign Reader at the subscription scope. What can the user see?

A. Every resource group and resource in that subscription (inheritance).

Q3. A user needs to manage VMs and also assign roles on them. Which single role fits?

A. Owner (Contributor can't assign roles).

Q4. Is 'Global Administrator' an Azure RBAC role or an Entra role?

A. An Entra (directory) role.

Q5. A user has Reader at a resource group and Contributor on one resource inside it. What's their effective access on that resource?

A. Contributor — RBAC is additive; the more specific assignment doesn't cancel the inherited one, it stacks.

CHAPTER SUMMARY

KEY TAKEAWAYS

- A role assignment = principal (who) + role definition (what) + scope (where).
- Owner = full + can grant access; Contributor = manage all but can't grant access; Reader = view. Narrow service-specific roles (VM Contributor, Network Contributor) are tightly scoped.
- Roles inherit down the hierarchy; RBAC is additive; deny overrides allow; use least privilege.
- Interpreting access means checking every scope that applies — use Check access or az role assignment list --all.
- Azure RBAC governs resources; Entra roles govern directory administration — separate systems entirely.

Subscriptions & Governance

WHAT YOU'LL LEARN

- Implement and manage Azure Policy
- Configure resource locks
- Apply and manage tags on resources
- Manage resource groups
- Manage subscriptions
- Manage costs by using alerts, budgets, and Azure Advisor recommendations
- Configure management groups

IN PLAIN ENGLISH

“Governance” is just a formal word for “how do we stop this from turning into a mess as more people and resources get added.” This chapter follows Microsoft’s official order: rules first (Policy), then protective controls (locks, tags), then the containers themselves (resource groups, subscriptions), then cost controls, and finally the broadest scale-out tool (management groups) last.

3.1 Implementing and managing Azure Policy

DEFINITION

Azure Policy: A governance service that evaluates your resources against rules and either audits non-compliant ones or prevents non-compliant deployments outright.

Where RBAC (Chapter 2) controls *who* can act, Azure Policy controls *what* is allowed to exist. Typical policies: allow only certain regions, require a tag, restrict VM sizes, or enforce encryption.

Policy effect	What it does
Deny	Blocks the non-compliant resource from being created/updated at all
Audit	Allows the resource, but flags it as non-compliant in the compliance report
Append	Adds a specified field/value to the resource during creation (e.g. force a tag) without blocking it
Modify	Adds, updates, or removes properties/tags on existing resources during creation or update
DeployIfNotExists (DINE)	After a resource is created, automatically deploys a related resource if it's missing
Disabled	The policy exists but does nothing — used for staged rollouts or testing

ANALOGY

RBAC is the bouncer checking IDs (who gets in). Azure Policy is the dress code (what's allowed once you're in) — even an Owner can be stopped from creating a resource that violates policy.

MEMORIZE THIS

RBAC = who can act; Azure Policy = what is allowed. Group policies into **initiatives** to manage them together.

EXAM TIP

Append vs Modify is a common trap: Append only works at creation time and can't touch existing resources; Modify can update properties on resources that already exist. "Add this tag going forward" = Append; "add this tag to everything, including what's already deployed" = Modify (often paired with a remediation task).

3.2 Configuring resource locks

Lock type	Effect
CanNotDelete	Resource can be modified but not deleted
ReadOnly	Resource can be read but not modified or deleted — including scaling a VM or resizing a disk

MEMORIZE THIS

A lock applied at a **resource group** is inherited by every resource inside it — even resources added later. Locks apply on top of RBAC: even an Owner can't delete a CanNotDelete-locked resource until the lock is removed.

3.3 Applying and managing tags

Tags are key/value labels (e.g. `env=prod`, `costcenter=42`) for organizing resources and reporting on cost by project or department.

MEMORIZE THIS

Tags are **NOT inherited by default** — tagging a resource group does not tag the resources inside it (a Policy with the Modify/Append effect is how you'd force that). A resource can have up to 50 tags; each name up to 512 characters, each value up to 256.

3.4 Managing resource groups

A **resource group** groups resources that share a lifecycle. A resource lives in exactly one resource group; deleting the RG deletes everything inside it. You can move resources between resource groups and even between subscriptions.

COMMAND EXAMPLE

```
# Move a resource to a different resource group
az resource move --destination-group newRG \
  --ids /subscriptions//resourceGroups/oldRG/providers/.../myResource
```

3.5 Managing subscriptions

A **subscription** is a billing and access boundary. Larger organizations use multiple subscriptions to separate environments (dev/test/prod), billing owners, or compliance boundaries. You can move resource groups between subscriptions (subject to provider and quota constraints in the destination).

MEMORIZE THIS

Everything in Azure is billed to exactly one subscription. Moving a subscription between management groups is a governance change, not a billing change — the invoice owner stays whoever it was unless separately transferred.

3.6 Managing costs: alerts, budgets, and Azure Advisor

Cloud spend can surprise you, so Azure gives you tools to watch and cap it. **Cost Management** analyzes spend; **budgets** set thresholds that trigger **alerts**; and **Azure Advisor** recommends ways to cut cost (and improve security, reliability, and performance).

MEMORIZE THIS

Budgets trigger **alerts** at spend thresholds — they notify, they don't auto-stop resources. **Azure Advisor** gives recommendations across cost, security, reliability, performance, and operational excellence. Use **tags** to break down cost by project/department.

3.7 Configuring management groups

Management groups sit above subscriptions and let you apply Azure Policy and RBAC to many subscriptions at once — the broadest, top-of-hierarchy governance tool, which is why Microsoft covers it last in this domain even though it sits highest in the hierarchy from Chapter 0.

MEMORIZE THIS

A management group lets you assign **Policy and RBAC once, at the top**, and have it flow down to every subscription and resource beneath it — the scale-out answer when a requirement says “apply this across all subscriptions.”

3.8 Hands-on lab: enforce tagging with Azure Policy

- **Step 1.** In the Azure portal, go to *Policy > Definitions* and search the built-in definition called **Require a tag on resource groups**.
- **Step 2.** Click *Assign*. Set the scope to one subscription or resource group (don't apply broadly on a shared subscription while testing). Set the tag name parameter to **costCenter**.
- **Step 3.** Under the *Non-compliance message* tab, add a clear message, then Review + create the assignment.
- **Step 4.** Try creating a new resource group **WITHOUT** the **costCenter** tag. If you assigned with effect Deny, the creation should be blocked with a policy violation message.
- **Step 5.** Go to *Policy > Compliance* and find your assignment — confirm it shows existing, untagged resource groups as non-compliant (this proves Audit-style visibility even before you touch enforcement).
- **Step 6.** Now add a **CanNotDelete** lock to one resource group from *Locks* in its left-hand menu, and try deleting the resource group — confirm Azure blocks it even though your account has Owner on the subscription.
- **Step 7 (cleanup).** Remove the lock, then remove the policy assignment once you've confirmed both behaviors.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. You must ensure no VM larger than a certain size is ever created. RBAC or Azure Policy?

A. Azure Policy (with a Deny effect) — it controls what's allowed to exist.

Q2. Which lock lets you change a resource but not delete it?

A. CanNotDelete.

Q3. Does a budget automatically shut down resources when exceeded?

A. No — it triggers alerts/notifications; it doesn't stop resources.

Q4. How do you apply governance to many subscriptions at once?

A. Use a management group and assign policy/RBAC there.

Q5. You need to add a tag to resources that already exist, not just new ones. Append or Modify?

A. Modify — Append only applies at creation time.

CHAPTER SUMMARY

KEY TAKEAWAYS

- Azure Policy controls what's allowed (Deny/Audit/Append/Modify/DeployIfNotExists); RBAC controls who can act.
- Resource locks (CanNotDelete/ReadOnly) are inherited from parent scope and apply on top of RBAC.
- Tags organize and enable cost reporting; not inherited by default.
- Resource groups hold resources sharing a lifecycle; subscriptions are the billing/access boundary.
- Cost Management + budgets + alerts track spend; Advisor recommends improvements.
- Management groups apply Policy/RBAC across many subscriptions at once — the broadest governance tool, covered last because it's the widest-scope wrap-up of everything above it.

PART 2

Storage

Configuring Access to Storage

WHAT YOU'LL LEARN

- Configure Azure Storage firewalls and virtual networks
- Create and use shared access signature (SAS) tokens
- Configure stored access policies
- Manage access keys
- Configure identity-based access for Azure Files

IN PLAIN ENGLISH

Before you even create a storage account in the next chapter, Microsoft's own guide wants you fluent in HOW access to storage is controlled — because every decision you make when actually creating an account (next chapter) depends on knowing these options first. Storage is reachable from the internet by default, so this chapter is a menu of increasingly precise ways to answer “who's allowed to read or write this data?”

4.1 Configuring Azure Storage firewalls and virtual networks

DEFINITION

Storage firewall: A network access control list on the storage account itself that restricts which networks (specific VNets/subnets, or public IP ranges) can even reach the account, regardless of what credentials a request presents.

By default a storage account accepts traffic from any network. The storage firewall lets you narrow that to only your VNets (via **service endpoints**, covered fully in the networking chapters) or specific public IP ranges — and you can set the default action to Deny so only explicitly allowed networks get through at all.

MEMORIZE THIS

A storage firewall restricts **where** requests can come from, independent of **who** is making them. Setting the default action to **Deny** and allowing only specific VNets/IPs is a standard “most secure” answer alongside private endpoints (also covered in the networking chapters).

4.2 Creating and using shared access signature (SAS) tokens

DEFINITION

Shared access signature (SAS): A signed URL that grants limited, time-bound access to specific storage resources — you control which services, which permissions, and an expiry time.

There are three distinct kinds of SAS, and the exam expects you to pick the right one for a scenario.

SAS type	Signed with	Scope
User delegation SAS	Entra ID credentials (most secure — no account key involved at all)	Blob storage only; tied to an Entra identity's permissions
Service SAS	The storage account key	One specific service (Blob, Files, Queue, or Table)
Account SAS	The storage account key	Can span MULTIPLE services at once and can grant account-level operations (like changing CORS rules)

ANALOGY

An access key is the master key to the whole building. A SAS token is a temporary visitor pass that opens only room 3, only until 5 PM, and only lets you look, not touch.

SAS permission letters

SAS permissions are shown as a compact string of letters: **read**, **write**, **delete**, **list**, **add**, **create**, **update**, **process**. Grant only the letters actually needed.

EXAM TIP

If a question stresses “no storage account key should ever be exposed, even indirectly,” the answer is a **user delegation SAS**. If the requirement spans multiple services (blob AND queue) in one token, that's an **account SAS**.

4.3 Configuring stored access policies

DEFINITION

Stored access policy: A server-side policy, defined on a container or file share, that a SAS token can reference instead of encoding its own permissions and expiry directly — letting you revoke or change many issued SAS tokens at once by editing or deleting the policy.

A raw SAS can't be revoked before it expires. Tying it to a stored access policy solves that: revoke or modify the policy and every SAS referencing it is immediately affected.

MEMORIZE THIS

A stored access policy is the mechanism for **centrally revoking** SAS tokens that were already issued — without it, a SAS is valid until its expiry no matter what.

4.4 Managing access keys

Every storage account has two **access keys** that grant full control over all its data. They exist in pairs so you can rotate one while the other stays in use. Because they're all-powerful, treat them like the crown jewels.

MEMORIZE THIS

Access keys give **full** access to the whole account. There are **two** so you can rotate without downtime. Never embed them in code; store them in **Key Vault** if you must use them. Prefer Entra ID + RBAC (4.5 and the identity-based methods) over keys wherever possible.

4.5 Configuring identity-based access for Azure Files

Azure Files traditionally required a storage account key or SAS to mount — but you can also authenticate with an **Entra ID** (or on-prem AD via Entra Connect) identity, so users mount a file share with the same credentials they use to sign in, and access is controlled with familiar Windows-style NTFS permissions on top of Azure RBAC for the share-level mount permission.

MEMORIZE THIS

Identity-based access for Azure Files needs BOTH an Azure RBAC role (e.g. Storage File Data SMB Share Contributor) to mount the share AND standard NTFS permissions on the actual files/folders — the two layers are independent and both must allow the action.

EXAM TIP

Most secure access ranking the exam expects you to reason about: **identity-based / Entra RBAC** (most secure, no secrets) > **user delegation SAS** > **service/account SAS** > **raw access keys** (least secure, full account power).

4.6 Hands-on lab: issue and test a service SAS

- **Step 1.** Create (or reuse) a storage account and a blob container named **evidence**. Upload a small test file to it.
- **Step 2.** In the portal, open the container and go to *Shared access tokens*. Set Permissions to **Read** and **List** only (uncheck everything else), set an expiry of 1 hour, and click *Generate SAS token and URL*.
- **Step 3.** Copy the Blob SAS URL and open it in a private/incognito browser window — confirm you can view/download the file with no Azure sign-in at all.
- **Step 4.** Edit the URL to remove the **sig=** parameter (the signature) and try again — confirm access is now denied, proving the signature is what Azure actually validates.
- **Step 5.** Go to the storage account's *Networking* blade, set *Public network access* to **Disabled from selected virtual networks and IP addresses**, and add your own current IP as an allowed range.
- **Step 6.** Retry the same SAS URL from a network NOT in your allow list (e.g. your phone on cellular data) and confirm it's now blocked — this proves the storage firewall is evaluated independently of, and in addition to, the SAS token's own permissions.
- **Step 7 (cleanup).** Reset the storage firewall to allow all networks (or leave it locked down if you're keeping this account for the next chapter's lab) and delete the test SAS token record if you created a stored access policy for it.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. You need to give a partner read-only access to one container for 24 hours. What do you issue?

A. A shared access signature (SAS) scoped to that container, read-only, expiring in 24 hours.

Q2. What lets you revoke many issued SAS tokens at once?

A. A stored access policy the SAS references.

Q3. Which SAS type is signed with Entra ID credentials instead of the storage account key?

A. A user delegation SAS.

Q4. What two independent layers control identity-based access to an Azure Files share?

A. Azure RBAC (to mount the share) and NTFS permissions (on the files/folders themselves).

Q5. Does a storage firewall care about the credentials in the request?

A. No — it restricts by network/source regardless of what credentials are presented.

CHAPTER SUMMARY

KEY TAKEAWAYS

- Storage firewalls restrict access by network, independent of credentials.
- Three SAS types: user delegation (Entra ID, most secure), service (one service), account (multiple services + account ops).
- Stored access policies let you centrally revoke issued SAS tokens.
- Access keys grant full account access; keep them in Key Vault, prefer more scoped methods.
- Identity-based access for Azure Files needs both RBAC (mount) and NTFS permissions (files) — independent layers.

Storage Accounts & Redundancy

WHAT YOU'LL LEARN

- Create and configure storage accounts
- Configure Azure Storage redundancy
- Configure object replication
- Configure storage account encryption
- Manage data by using Azure Storage Explorer and AzCopy

IN PLAIN ENGLISH

Now that you know how access to storage is controlled (Chapter 4), this chapter is about the account itself — think of it as an unlimited external hard drive that Microsoft looks after for you. The real decisions here are: how many copies of your data, and how far apart should they be kept?

5.1 Creating and configuring storage accounts

DEFINITION

Storage account: A container that provides access to Azure Storage data services — Blob, Files, Queue, and Table — under one namespace, with shared settings like redundancy and encryption.

Detail	Exact rule
Storage account name	3–24 characters, lowercase letters and numbers ONLY, globally unique across all of Azure
Performance tiers	Standard (HDD-backed, general purpose) vs Premium (SSD-backed, low-latency)
Account kind	StorageV2 (general purpose v2) is the default/recommended for almost everything

5.2 Configuring Azure Storage redundancy

Azure always keeps multiple copies of your data. The redundancy option decides how many and how far apart, which determines what kind of failure you can survive.

Option	Copies / spread	Survives
LRS	3 copies in one datacenter	Disk/hardware failure only
ZRS	3 copies across availability zones in the region	A whole datacenter/zone failure
GRS	LRS locally + 3 async copies in a paired region	A full regional outage (via failover)
GZRS	ZRS locally + async copies in a paired region	Zone failure AND regional outage
RA-GRS / RA-GZRS	GRS/GZRS + read access to the secondary region	Same, plus read access to the secondary anytime

MEMORIZE THIS

LRS=Local, ZRS=Zones, GRS=Geo, GZRS=zones+geo. **RA-** adds read access to the secondary. GRS failover to the secondary is a **manual, customer-initiated action** (or Microsoft-initiated in a true disaster) — it does not happen automatically.

EXAM TIP

An app needs to keep working, unmodified, if the primary region goes down — RA-GRS alone does NOT satisfy this. The app must be coded to detect failure and switch to the **-secondary** endpoint, or you must perform an account failover.

5.3 Configuring object replication

DEFINITION

Object replication: Asynchronous copying of block blobs from a source storage account to a destination account, optionally in a different region — used to bring data closer to users or maintain a separate copy for read isolation.

MEMORIZE THIS

Object replication is account-to-account (often cross-region) for a specific purpose like latency or read isolation — different from redundancy, which is Azure's automatic behind-the-scenes copying within the SAME account.

5.4 Configuring storage account encryption

All Azure Storage is **encrypted at rest by default** using Microsoft-managed keys — you don't turn it on. You can optionally supply your own keys (customer-managed keys in Key Vault) for more control over key rotation and revocation.

MEMORIZE THIS

Storage encryption at rest is **always on** by default (Microsoft-managed keys); customer-managed keys are optional, not required to satisfy “is data encrypted” questions.

5.5 Managing data with Storage Explorer and AzCopy

Tool	What it's for
Azure Storage Explorer	A GUI app to browse and manage storage data visually
AzCopy	A command-line tool for fast, scriptable bulk copy/move of data

COMMAND EXAMPLE

```
# Copy a local folder to a blob container with AzCopy
azcopy copy './data' 'https://acct.blob.core.windows.net/mycontainer' --recursive
```

5.6 Hands-on lab: configure GRS and observe the failover process

- **Step 1.** Create a new storage account (or edit an existing one) and set *Redundancy* to **Geo-redundant storage (GRS)** during creation — this can't always be changed after the fact without a support request, so set it at creation for this lab.
- **Step 2.** Upload a test blob, then go to the storage account's *Geo-replication* blade and confirm the *Last sync time* field — this shows how current the secondary region actually is (RPO in practice, tying back to Chapter 15's RPO/RTO material).
- **Step 3.** Note the read-only secondary endpoint shown on that same blade (**-secondary** suffix) — if you'd enabled RA-GRS instead, you could browse this endpoint directly.
- **Step 4 (read-only, do not actually execute in a real subscription unless testing DR deliberately).** The *Prepare for failover* button on this blade is what actually performs a manual failover — note that it is a customer-triggered action, not automatic, confirming the exam fact from 5.2.
- **Step 5 (cleanup).** If you don't need ongoing geo-redundancy for later labs, you can downgrade to LRS afterward to reduce cost.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. Which redundancy option protects against a full region going down but is the cheapest that does so?

A. GRS (geo-redundant).

Q2. Do you need to enable encryption at rest for a storage account?

A. No — it's on by default with Microsoft-managed keys.

Q3. Which tool would you script to bulk-upload thousands of files?

A. AzCopy.

Q4. Is a GRS failover to the secondary region automatic?

A. No — it's a manual, customer-initiated action (or Microsoft-initiated only in a true disaster).

Q5. What's the difference between redundancy and object replication?

A. Redundancy is Azure's automatic copying within the same account; object replication is a deliberate account-to-account copy, often cross-region, for latency or read isolation.

CHAPTER SUMMARY

KEY TAKEAWAYS

- Storage account names: 3–24 chars, lowercase/numbers only, globally unique.
- Redundancy ladder: LRS → ZRS → GRS → GZRS; RA- adds read access to the secondary; GRS failover is manual.
- Object replication is a deliberate account-to-account copy; different from automatic redundancy.
- Encryption at rest is always on by default; customer-managed keys are optional.
- Storage Explorer = GUI; AzCopy = scriptable bulk transfer.

Blob Storage & Azure Files

WHAT YOU'LL LEARN

- Create and configure a file share in Azure Files
- Create and configure a container in Azure Blob Storage
- Configure storage tiers
- Configure soft delete for blobs and containers
- Configure snapshots and soft delete for Azure Files
- Configure blob lifecycle management
- Configure blob versioning

IN PLAIN ENGLISH

With access (Ch 4) and the account itself (Ch 5) covered, this chapter is where you actually put data in. Two questions drive it: what kind of thing am I storing (a proper shared folder, or a flexible bucket of objects), and how often will I actually look at it?

6.1 Creating and configuring a file share in Azure Files

DEFINITION

Azure Files: Fully managed SMB/NFS file shares you can mount like a network drive from Windows, Linux, or macOS.

Use Files when you want a shared drive that multiple machines mount the traditional way — identity-based access (Chapter 4.5) layers on top of this for Entra-integrated permissions.

6.2 Creating and configuring a container in Azure Blob Storage

DEFINITION

Azure Blob Storage: Object storage for unstructured data (files, images, backups), organized into containers and accessed over HTTPS/REST, SDKs, or SAS.

Use Blob when apps read/write objects over HTTP — a photo store, a backup target, a data lake landing zone.

6.3 Configuring storage tiers

Tier	Use for	Minimum stay
Hot	Frequently accessed data	None
Cool	Infrequent access	30 days
Cold	Rare access	90 days
Archive	Almost never accessed — OFFLINE	180 days

MEMORIZE THIS

Tiers hot to cold: **Hot** → **Cool** → **Cold** → **Archive**. **Archive is offline** — reading requires rehydration first. Moving a blob out of a tier before its minimum stay is up still succeeds, but Azure bills an early-deletion penalty as if it had stayed the full period.

Rehydrating from Archive

Rehydration priority	Typical time
Standard	Up to 15 hours
High	Under 1 hour for blobs under 10 GB (best-effort, no guaranteed SLA)

6.4 Configuring soft delete for blobs and containers

Feature	What it does
Blob soft delete	Keeps deleted blobs recoverable for a retention window
Container soft delete	Keeps deleted containers (and everything in them) recoverable

6.5 Configuring snapshots and soft delete for Azure Files

Azure Files has its own equivalent protections: **snapshots** are read-only point-in-time copies of a share, and **share soft delete** keeps a deleted share recoverable for a retention window — the same concepts as 6.4, implemented separately for Files.

MEMORIZE THIS

Blob protections (6.4) and File share protections (6.5) are configured in *different places* in the portal even though the concepts mirror each other — don't assume enabling one enables the other.

6.6 Configuring blob lifecycle management

DEFINITION

Lifecycle management: Rules that automatically move blobs to cooler tiers or delete them based on age or last-access time.

COMMAND EXAMPLE

```
# Example rule intent (set as JSON policy in the portal)
# If a blob hasn't been modified in 30 days -> move to Cool
# If not modified in 180 days -> move to Archive; after 365 -> delete
```

6.7 Configuring blob versioning

Blob versioning automatically keeps previous versions when a blob is overwritten, so an accidental overwrite is always recoverable — distinct from a manually-triggered snapshot.

MEMORIZE THIS

Soft delete recovers from deletion. **Versioning** recovers from an overwrite. **Snapshots** are manually-triggered point-in-time copies. All three can run together.

6.8 Hands-on lab: build a full data-protection stack on one container

- **Step 1.** On the storage account from Chapter 5's lab, go to *Data protection* and enable **Enable soft delete for blobs** (7-day retention) and **Enable versioning for blobs**.
- **Step 2.** Upload a text file to your container, note its contents, then overwrite it with different content through the portal's upload-and-replace option.
- **Step 3.** Open the blob's *Version history* tab — confirm the original version is still there and downloadable, proving versioning caught the overwrite automatically.

- **Step 4.** Delete the blob entirely, then go to the container and toggle **Show deleted blobs** — confirm it's still visible and can be undeleted, proving soft delete caught the deletion.
- **Step 5.** Create a lifecycle management rule (*Lifecycle management > Add a rule*) that moves blobs to Cool after 30 days of no modification, scoped to your test container.
- **Step 6 (cleanup).** Undelete the test blob, delete the lifecycle rule if you don't want it running long-term, and leave versioning/soft delete on — you'll want them for later chapters.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. You need a shared drive several VMs can mount like a network folder. Blob or Files?

A. Azure Files.

Q2. Which tier is cheapest to store but must be rehydrated before reading?

A. Archive.

Q3. A blob was overwritten by mistake. Which feature lets you recover the previous content?

A. Blob versioning (or a snapshot / soft delete).

Q4. How do you automatically move 90-day-old data to a cheaper tier?

A. A blob lifecycle management rule.

Q5. Do blob soft delete settings automatically also protect Azure Files shares?

A. No — they're configured separately even though the concepts mirror each other.

CHAPTER SUMMARY

KEY TAKEAWAYS

- Blob = object storage in containers; Files = mountable SMB/NFS shares.
- Tiers Hot/Cool/Cold/Archive trade storage vs access cost; Archive is offline (rehydrate: Standard ~15h, High <1h best-effort).
- Soft delete (blobs and containers) protects against deletion; configured separately for Files (snapshots + share soft delete).
- Lifecycle management auto-tiers/deletes by age; versioning protects against overwrites.

PART 3

Compute

Infrastructure as Code: ARM & Bicep

WHAT YOU'LL LEARN

- Interpret an Azure Resource Manager template or a Bicep file
- Modify an existing Azure Resource Manager template
- Modify an existing Bicep file
- Deploy resources by using an ARM template or a Bicep file
- Export a deployment as an ARM template, or convert ARM to Bicep

IN PLAIN ENGLISH

Everything you've built so far you could do by clicking through the portal. This chapter introduces writing down what you want in a text file, then handing that file to Azure and letting it build exactly what's described — the difference between describing a room to a builder from memory every time versus handing them a blueprint.

7.1 Interpreting an ARM template or Bicep file

	ARM template	Bicep
Format	JSON — verbose	A clean, purpose-built language
Relationship	The underlying format Azure accepts	Transpiles (compiles) down to ARM JSON
Behavior	Declarative & idempotent	Declarative & idempotent

A REAL TEMPLATE, START TO FINISH

```
# A complete, tiny Bicep file that creates one storage account
param storageName string
param location string = resourceGroup().location
resource stg 'Microsoft.Storage/storageAccounts@2023-01-01' = {
  name: storageName
  location: location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
}
```

IN PLAIN ENGLISH

Nine short lines describe an entire storage account. The first two lines are **parameters** (blanks filled in at deploy time). The **resource** block is the thing being created. There's no step-by-step instructions anywhere — you describe the end result, and Azure figures out how to get there. That's “declarative.”

DEFINITION

Idempotent: Deploying the same template repeatedly produces the same end state — existing correct resources are left as-is, not duplicated.

MEMORIZE THIS

Bicep transpiles to ARM JSON — same result, friendlier syntax. A template has four functional sections: **parameters**, **variables**, **resources**, **outputs**.

7.2 Modifying an existing ARM template

Editing raw ARM JSON means finding the right **resources** array entry and adjusting its **properties** block — tedious but sometimes necessary when you only have the JSON and not the original Bicep source.

7.3 Modifying an existing Bicep file

The same edit is far more readable in Bicep — change the property directly on the **resource** block, or add a new **param** if the value should be configurable at deploy time instead of hardcoded.

7.4 Deploying resources with an ARM template or Bicep file

Deployment mode	Behavior on resources NOT in the template but present in the RG
Incremental (default)	Leaves them alone — only adds/updates what's in the template
Complete	DELETES them — the resource group ends up matching the template exactly

COMMAND EXAMPLE

```
# Deploy a Bicep or ARM file to a resource group
az deployment group create --resource-group myRG --template-file main.bicep
```

EXAM TIP

Complete mode is destructive and a favorite trap: a question describing unexpected resource deletions after a deployment is almost always someone using **Complete** mode by mistake. Incremental is the safe default.

7.5 Exporting a deployment, or converting ARM to Bicep

COMMAND EXAMPLE

```
# Convert an existing ARM template to Bicep
az bicep decompile --file template.json
# Export a deployed resource group as a template (portal: Export template)
```

MEMORIZE THIS

Know the directions: **decompile** = ARM → Bicep; **build** = Bicep → ARM. **Export template** captures existing resources as a template you can reuse.

7.6 Hands-on lab: deploy, modify, and redeploy a Bicep file

- **Step 1.** Save the 9-line Bicep example from 7.1 to a file named **storage.bicep** locally or in Cloud Shell.
- **Step 2.** Run a what-if to preview the deployment before committing: **az deployment group what-if --resource-group myRG --template-file storage.bicep --parameters storageName=mydemostg123**.
- **Step 3.** Deploy for real with **az deployment group create** using the same parameters, and confirm the storage account appears in the portal.
- **Step 4.** Edit the Bicep file to add a new line inside the resource block: **properties: { minimumTlsVersion: 'TLS1_2' }**, then redeploy the exact same command — confirm it updates the existing account instead of creating a duplicate (idempotency in action).
- **Step 5.** From the portal, open the resource group and use *Export template* to download the ARM JSON Azure generated — compare its length and readability to your Bicep source.
- **Step 6 (cleanup).** Delete the storage account if you don't need it for later chapters.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. What does Bicep compile down to?

A. ARM template JSON.

Q2. What does 'idempotent' mean for a deployment?

A. Re-running it yields the same end state without duplicating existing resources.

Q3. How do you turn an existing ARM template into Bicep?

A. Decompile it (az bicep decompile).

Q4. You deploy in Complete mode. What happens to resources in the RG that aren't in the template?

A. They get deleted.

CHAPTER SUMMARY

KEY TAKEAWAYS

- IaC describes resources in versionable text for repeatable deployments; a template has parameters/variables/resources/outputs.
- Bicep is friendlier and transpiles to ARM JSON; both are declarative and idempotent.
- Incremental (default, safe) vs Complete (deletes extras) deployment modes.
- Decompile ARM→Bicep, build Bicep→ARM, Export template captures existing resources.

Virtual Machines & Availability

WHAT YOU'LL LEARN

- Create a virtual machine
- Configure encryption at host for Azure virtual machines
- Move a virtual machine to another resource group, subscription, or region
- Manage virtual machine sizes
- Manage virtual machine disks
- Deploy virtual machines to availability zones and availability sets
- Deploy and configure an Azure Virtual Machine Scale Set

IN PLAIN ENGLISH

A virtual machine is the most familiar thing in this whole book: it's just a computer — operating system, disk, CPU, memory — except it lives in a Microsoft datacenter and you can create, resize, or delete it in seconds.

8.1 Creating a virtual machine

When you create a VM you choose its **size** (the family/SKU that sets CPU and memory), its **disks**, its network, and its region.

COMMAND EXAMPLE

```
# Create a Linux VM with the CLI
az vm create --resource-group myRG --name myVM \
  --image Ubuntu2204 --size Standard_B2s --admin-username azureuser \
  --generate-ssh-keys
```

VM extensions

DEFINITION

VM extension: A small agent installed inside the guest OS after the VM is running, that performs post-deployment configuration, automation, or monitoring — without you having to log in and do it by hand.

Extension	What it does
Custom Script Extension	Runs a script you provide (PowerShell/Bash) at deploy time or on demand
Azure Monitor Agent (AMA)	Ships metrics/logs to Azure Monitor — required for VM Insights (Chapter 14)
Microsoft Antimalware	Installs and manages built-in antimalware protection
Desired State Configuration (DSC)	Applies and enforces a declared PowerShell configuration state

MEMORIZE THIS

Extensions run **inside the guest OS**, which is why they need the VM to actually be running — unlike disk/network settings which are configured at the Azure Resource Manager layer regardless of the VM's power state.

8.2 Configuring encryption at host

Encryption at host encrypts VM disk data — including temp disk and caches — on the host itself, rather than relying only on the guest OS's own disk encryption.

MEMORIZE THIS

Encryption at host must be **registered as a feature** on your subscription before use ([az feature register](#)) — it isn't on by default.

8.3 Moving a virtual machine

You can move a VM to another resource group, subscription, or even region. Moving between resource groups/subscriptions is a metadata operation; moving *regions* actually requires redeployment (via Azure Resource Mover or a manual rebuild), since a VM's disks are region-bound.

EXAM TIP

“Move” to another resource group or subscription is straightforward (Resource Manager just re-parents it). “Move” to another **region** is NOT the same operation — it requires a migration tool or redeployment, since compute and storage resources don't span regions.

8.4 Managing virtual machine sizes

VM size/SKU sets CPU + memory. You can resize a VM after creation, but typically only while it's **stopped (deallocated)** — and only to a size available in the same hardware cluster; some resizes require a redeploy.

8.5 Managing virtual machine disks

Disk tier	Performance/cost
Standard HDD	Lowest cost, lowest performance
Standard SSD	Balanced cost/performance
Premium SSD	High performance, required for most production and for the 99.9% single-VM SLA
Ultra Disk	Highest performance, configurable IOPS/throughput independent of size

MEMORIZE THIS

OS disk + optional data disks; the temporary/local disk is **not persistent** — never store anything there you can't afford to lose on reboot/reallocation.

8.6 Deploying VMs to availability zones and availability sets

Option	Protects against	How
Availability Set	Hardware failures & maintenance WITHIN one datacenter	Spreads VMs across fault domains and update domains
Availability Zones	An entire datacenter failure	Spreads VMs across physically separate zones in the region

DEFINITION

Fault domain: A group of hardware sharing power and network — a single point of physical failure. Update domain: a group rebooted together during planned maintenance.

Configuration	SLA
Single VM, Standard HDD	No SLA guarantee

Configuration	SLA
Single VM, Standard SSD	~99.5%
Single VM, Premium SSD/Ultra for ALL disks	99.9%
2+ VMs in an Availability Set	99.95%
2+ VMs across 2+ Availability Zones	99.99%

MEMORIZE THIS

A single VM only reaches 99.9% if EVERY disk is Premium SSD or Ultra Disk. You can only place a VM into an availability set at creation time — you cannot add a running VM to one afterward. Not every VM size supports Availability Zones in every region.

8.7 Deploying and configuring a Virtual Machine Scale Set

DEFINITION

Virtual Machine Scale Set (VMSS): A group of identical VMs that automatically scale out (add instances) or in (remove them) based on rules or a schedule.

MEMORIZE THIS

Scale up = bigger VM size (vertical). **Scale out** = more instances (horizontal, what VMSS does). VMSS is the answer whenever a scenario needs elastic capacity behind a load balancer.

8.8 Hands-on lab: deploy two VMs into an availability set

- **Step 1.** Create an availability set first: *Create a resource > Availability set*, 5 update domains, 2 fault domains (the defaults).
- **Step 2.** Create a B-series VM and, on the *Basics* tab under *Availability options*, choose **Availability set** and select the one you just created. Use Premium SSD for the OS disk if your subscription allows it, to see the 99.9%-eligible configuration.
- **Step 3.** Repeat step 2 for a second VM, placed in the SAME availability set.
- **Step 4.** Open the availability set resource and check the *Overview* — confirm both VMs are listed and note which fault domain/update domain each landed in.
- **Step 5.** Try to add a THIRD, already-existing VM (one you created earlier without an availability set) to this set after the fact — confirm the portal does not offer this option, proving the “set at creation only” rule.
- **Step 6 (cleanup).** Deallocate (not just stop) both VMs when you're done to avoid ongoing compute charges, or delete them if you don't need them for later labs.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. You need VMs to survive an entire datacenter outage in a region. Set or zones?

A. Availability Zones.

Q2. What does an availability set spread VMs across?

A. Fault domains and update domains, within a single datacenter.

Q3. Which scaling adds more identical VM instances automatically?

A. Scale out, using a Virtual Machine Scale Set.

Q4. Can you resize a running VM to any size at any time?

A. Typically only while deallocated, and only to a size available in the same cluster.

Q5. Is moving a VM to another region the same operation as moving it to another resource group?

A. No — cross-region requires migration/redeployment; RG/subscription moves are metadata operations.

CHAPTER SUMMARY

KEY TAKEAWAYS

- VM size sets CPU/RAM; disk tiers trade cost vs performance (HDD→SSD→Premium→Ultra).
- Encryption at host needs feature registration first.
- Moving RG/subscription is simple; moving regions requires redeployment.
- Availability Set = fault/update domains within a datacenter (99.95%); Zones = across datacenters (99.99%); single VM needs all-Premium/Ultra disks for 99.9%.
- VMSS auto-scales identical VMs; scale up = bigger, scale out = more.

Containers in Azure

WHAT YOU'LL LEARN

- Create and manage an Azure Container Registry
- Provision a container by using Azure Container Instances
- Provision a container by using Azure Container Apps
- Manage sizing and scaling for containers

IN PLAIN ENGLISH

A developer packages up their application AND everything it needs to run into one self-contained file called an **image**. That image runs identically anywhere, because it doesn't rely on whatever happens to already be installed on the host — it brought its own.

9.1 Creating and managing Azure Container Registry

DEFINITION

Azure Container Registry (ACR): A private store for your container images.

ACR tier	Key difference
Basic	Lower storage/throughput; fine for dev/test
Standard	More storage/throughput for production workloads
Premium	Adds geo-replication, content trust (image signing), and private endpoint support

MEMORIZE THIS

Only Premium ACR supports **geo-replication** — one registry, low-latency across multiple regions.

9.2 Provisioning a container with Azure Container Instances

DEFINITION

Azure Container Instances (ACI): The fastest way to run a single container: serverless, no orchestration.

COMMAND EXAMPLE

```
# Push then run a container image
az acr build --registry myACR --image app:v1 .
az container create --resource-group myRG --name c1 \
  --image myACR.azurecr.io/app:v1 --cpu 1 --memory 1.5
```

Restart policy	Behavior
Always	Restarts no matter how it exits (default)
Never	Runs once and stops, regardless of exit code
OnFailure	Restarts only if it exits with an error

MEMORIZE THIS

For a **one-off batch job** in ACI, use **Never** or **OnFailure** — the default **Always** would keep re-running it forever.

9.3 Provisioning a container with Azure Container Apps

DEFINITION

Azure Container Apps: Serverless containers with autoscaling, microservices support, and revisions — built on Kubernetes, but you don't manage the cluster.

EXAM TIP

For a simple one-off container → **ACI**. For a scalable production app without running Kubernetes yourself → **Container Apps**. Full Kubernetes control (you manage the cluster) is AKS, not covered by this exam's objectives.

9.4 Managing sizing and scaling for containers

Sizing is set by CPU/memory per container in both ACI and Container Apps. Container Apps additionally scales on rules — HTTP traffic, queue length, CPU — including **scale-to-zero** when idle, which ACI does not do (an ACI container runs until you stop it).

MEMORIZE THIS

Container Apps can scale to zero (no cost while idle); **ACI runs until explicitly stopped** and bills for that whole time. This distinction is a common cost-optimization scenario question.

9.5 Hands-on lab: build and run a container end to end

- **Step 1.** Create a Basic-tier ACR: `az acr create --resource-group myRG --name myuniqueacr123 --sku Basic`.
- **Step 2.** Build and push a minimal image directly in ACR (no local Docker needed) using a public sample: `az acr build --registry myuniqueacr123 --image helloworld:v1 https://github.com/Azure-Samples/aci-helloworld.git`.
- **Step 3.** Run it as a one-off container instance: `az container create --resource-group myRG --name hello-aci --image myuniqueacr123.azurecr.io/helloworld:v1 --cpu 1 --memory 1 --ports 80 --ip-address Public`, using credentials from `az acr credential show`.
- **Step 4.** Browse to the public IP shown in the output and confirm the sample app loads.
- **Step 5.** Deploy the same image as a Container App instead (*Create a resource > Container App*, point it at your ACR image) and enable autoscale with a minimum replica count of 0 — confirm in the Container App's metrics that it scales down to zero replicas after a period of no traffic, which the ACI version never does.
- **Step 6 (cleanup).** Delete the container instance, the Container App, and the ACR when done.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. Where do you store your private container images in Azure?

A. Azure Container Registry (ACR).

Q2. You need the quickest way to run a single container with no orchestration. Which service?

A. Azure Container Instances (ACI).

Q3. Which service runs containers that autoscale (even to zero) without you managing Kubernetes?

A. Azure Container Apps.

Q4. Which ACR tier is required for geo-replication?

A. Premium.

Q5. Does ACI scale to zero when idle?

A. No — it runs (and bills) until explicitly stopped; only Container Apps scales to zero.

CHAPTER SUMMARY

KEY TAKEAWAYS

- ACR stores images; only Premium tier supports geo-replication.
- ACI runs a single container fast with no orchestration; restart policies control re-run behavior.
- Container Apps runs autoscaling serverless containers (built on Kubernetes, unmanaged by you) and can scale to zero; ACI cannot.
- Size by CPU/memory per container in both services.

Azure App Service

WHAT YOU'LL LEARN

- Provision an App Service plan
- Configure scaling for an App Service plan
- Create an App Service
- Configure certificates and TLS for an App Service
- Map an existing custom DNS name to an App Service
- Configure backup for an App Service
- Configure networking settings for an App Service
- Configure deployment slots for an App Service

IN PLAIN ENGLISH

Sometimes you don't want to manage a VM or a container at all — you just have a website or API's code, and you want to upload it and have it work, with Azure handling the server underneath. That's App Service.

10.1 Provisioning an App Service plan

Plan tier	What it unlocks
Free / Shared	Dev/test only; shares compute; no custom domain TLS on Free
Basic	Dedicated compute, custom domains + TLS, manual scale out only
Standard	Adds deployment slots (5), autoscale, daily backups
Premium	More slots (20), faster hardware, VNet integration
Isolated	Runs in your own dedicated App Service Environment (ASE) for full network isolation

10.2 Configuring scaling for an App Service plan

Scaling	Means	How
Scale up	Move to a bigger/more powerful plan tier	Change the plan SKU (vertical)
Scale out	Add more instances of the app	Manual or autoscale on Standard+ (horizontal)

MEMORIZE THIS

Autoscale requires Standard tier or higher. Basic can scale out manually but cannot autoscale on a rule/schedule.

10.3 Creating an App Service

DEFINITION

Azure App Service: A fully managed PaaS platform for hosting web apps, REST APIs, and backends — you deploy your code and Azure handles the servers, patching, and scaling.

Every App Service runs on a plan (10.1), which sets the compute it uses and, by tier, which of the features below are even available.

10.4 Configuring certificates and TLS

Bind a TLS/SSL certificate to your App Service to secure HTTPS traffic — use an App Service Managed Certificate (free, auto-renewing) for a custom domain, or bring your own certificate for more control.

10.5 Mapping a custom DNS name

Point your own domain at the App Service by adding a CNAME (or A record + TXT verification record) at your DNS provider, then adding the custom domain in the App Service's *Custom domains* blade.

10.6 Configuring backup for an App Service

Automatic backups of the app and its content are available on Standard tier and above — configure a schedule and retention, and back up to a storage account.

10.7 Configuring networking settings

Integrate the App Service with a VNet, restrict inbound access, and use private endpoints to reach it without traversing the public internet — covered in depth alongside private endpoints for other PaaS services in the networking chapters.

10.8 Configuring deployment slots

DEFINITION

Deployment slot: A separate, live instance of your app with its own URL, which you can deploy to and test, then **swap** into production with no downtime.

ANALOGY

A deployment slot is like rehearsing a play on a second identical stage. When it's perfect, you swap the stages instantly — the audience never sees a gap.

A swap isn't an instant DNS flip — Azure warms up the target slot's app in the source slot's container first, waits for a successful response, and only then redirects traffic. That's what makes it zero-downtime.

MEMORIZE THIS

Deployment slots require Standard tier or higher. Slot-specific settings (marked “deployment slot setting”) stay with the slot and do NOT swap — staging keeps pointing at a staging database even after a swap, while everything else moves.

10.9 Hands-on lab: deploy an app with a staging slot and swap

- **Step 1.** Create an App Service on a **Standard S1** plan (needed for slots) using any quickstart language runtime.
- **Step 2.** Under *Deployment slots*, click *Add Slot*, name it **staging**, and clone settings from the production slot.
- **Step 3.** On the staging slot's *Configuration* blade, add an app setting named **ENVIRONMENT = staging**, and toggle the *Deployment slot setting* checkbox on it — this makes it slot-sticky.
- **Step 4.** Deploy any sample app content to the staging slot's URL (shown on its Overview page) and confirm it loads.
- **Step 5.** Go to production's *Deployment slots* blade and click *Swap*, staging → production. After it completes, check the (now production) app's environment setting.
- **Step 6.** Confirm the **ENVIRONMENT** setting did NOT swap with the content — the slot-specific setting stayed behind, exactly as the exam fact predicts.
- **Step 7 (cleanup).** Delete the staging slot and scale the plan back down if you don't need it running.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. What defines the compute size, tier, and cost your web app runs on?

A. The App Service plan.

Q2. You need to release a new version with no downtime and easy rollback. What do you use?

A. Deployment slots (deploy to staging, then swap).

Q3. Adding more instances of the app is scaling up or scaling out?

A. Scaling out (horizontal).

Q4. What tier is required for deployment slots and autoscale?

A. Standard or higher.

Q5. A staging slot has a slot-specific connection string. After a swap to production, which connection string does production use?

A. Its own — slot-specific settings don't swap; they stay with the slot.

CHAPTER SUMMARY

KEY TAKEAWAYS

- App Service plan tiers set price, features, and scaling capability (Standard+ needed for slots and autoscale).
- Scale up = bigger plan; scale out = more instances.
- Certificates/TLS secure custom domains; DNS mapping points your domain at the app.
- Backup (Standard+), VNet integration/private endpoints for networking.
- Deployment slots give zero-downtime swaps; slot-specific settings deliberately don't move during a swap.

PART 4

Networking

Virtual Networks & Connectivity

WHAT YOU'LL LEARN

- Create and configure virtual networks and subnets
- Create and configure virtual network peering
- Configure public IP addresses
- Configure user-defined routes
- Troubleshoot network connectivity

IN PLAIN ENGLISH

Networking is the topic most likely to feel intimidating if you've never worked with IP addresses before. One piece of notation you'll see constantly: **10.0.0.0/16**. The first part is a starting IP address. The **/16** says how many addresses belong to the range — a SMALLER number after the slash means MORE addresses (a /16 gives about 65,000; a /24 gives 256).

11.1 Creating and configuring virtual networks and subnets

DEFINITION

Virtual network (VNet): Your own private, isolated network in Azure, defined by an address space, within which resources like VMs communicate securely.

A VNet is subdivided into **subnets** — smaller ranges that group and isolate resources.

COMMAND EXAMPLE

```
# Create a VNet with a subnet
az network vnet create --resource-group myRG --name myVNet \
  --address-prefix 10.0.0.0/16 --subnet-name web --subnet-prefix 10.0.1.0/24
```

- **x.x.x.0** — the network address
- **x.x.x.1** — reserved for the default gateway
- **x.x.x.2** and **x.x.x.3** — reserved to map Azure DNS IPs into the subnet
- **x.x.x.255** (last address) — the broadcast address

MEMORIZE THIS

Azure reserves **5 IP addresses per subnet**. A /24 subnet has 256 addresses total but only **251 usable**. Resources in a VNet communicate by default; VNets are isolated from each other unless connected.

11.2 Creating and configuring virtual network peering

To let resources in two separate VNets communicate, create **VNet peering** — a low-latency connection over Microsoft's backbone, regional or global.

EXAM TIP

Peering is NOT transitive. If VNet A peers with B, and B peers with C, A still can't reach C without a direct A–C peering or a hub/gateway design. Peered VNets must not have overlapping address spaces.

11.3 Configuring public IP addresses

Concept	What to know
Public IP SKU	Basic vs Standard; Standard is secure-by-default (closed until an NSG allows) and zone-redundant
Static vs dynamic	Static keeps the same IP; dynamic can change on deallocation

11.4 Configuring user-defined routes

System routes	Azure's automatic default routing between subnets and to the internet
User-defined route (UDR)	A custom route you add to override defaults — e.g. force traffic through a firewall
Next hop type	Sends traffic to
Virtual appliance	A specific IP address, typically a firewall NVA
Virtual network gateway	A VPN/ExpressRoute gateway
VNet peering	Directly to a peered VNet
Internet	The public internet (default for most traffic)
None	Drops the traffic entirely — a deliberate black hole

MEMORIZE THIS

UDRs override system routes — commonly to force subnet traffic through a firewall (next hop). Next hop type “None” is a legitimate answer for deliberately blocking traffic at the routing layer.

11.5 Troubleshooting network connectivity

When traffic won't flow between resources, check in order: is the NSG on either end allowing it? Are effective routes correct (a UDR might be redirecting traffic somewhere unexpected)? Is there overlapping address space breaking a peering? **Network Watcher** tools (covered fully in the Monitoring chapters) — IP flow verify, next hop, Connection Monitor — pinpoint exactly where traffic is blocked rather than guessing.

MEMORIZE THIS

For connectivity problems, check **NSG rules** and **effective routes** first — these two cover the large majority of “why can't A reach B” scenarios before you reach for deeper diagnostic tools.

11.6 Hands-on lab: peer two VNets and prove non-transitivity

- **Step 1.** Create three VNets: **vnet-a** (10.10.0.0/16), **vnet-b** (10.20.0.0/16), **vnet-c** (10.30.0.0/16), each with one subnet and one small test VM.
- **Step 2.** Peer **vnet-a** ↔ **vnet-b** (bidirectional, both directions must be created), then peer **vnet-b** ↔ **vnet-c**.
- **Step 3.** From the VM in vnet-a, ping (or Test-NetConnection) the private IP of the VM in vnet-b — confirm it succeeds.
- **Step 4.** From the same VM in vnet-a, try to reach the VM in vnet-c — confirm it FAILS, proving peering isn't transitive even though a path exists through vnet-b.
- **Step 5.** Add a direct peering vnet-a ↔ vnet-c and retry step 4 — confirm it now succeeds.
- **Step 6 (cleanup).** Remove all three peerings and delete the VNets/VMs if not needed for later labs.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. VNet A peers with B, B peers with C. Can A reach C?

A. No — peering isn't transitive; you need direct peering or a hub/gateway.

Q2. How do you force a subnet's traffic through a firewall appliance?

A. A user-defined route (UDR) with the firewall as next hop.

Q3. How many IP addresses does Azure reserve in each subnet?

A. Five (the first four and the last).

Q4. What are the first two things to check when troubleshooting connectivity between two resources?

A. NSG rules and effective routes.

CHAPTER SUMMARY

KEY TAKEAWAYS

- A VNet is a private network; subnets segment it (Azure reserves 5 IPs per subnet).
- VNet peering connects VNets over the backbone but is non-transitive; no overlapping address spaces.
- Standard public IPs are closed by default and need an NSG rule to allow inbound.
- UDRs override default routing; next hop "None" deliberately blocks traffic.
- Troubleshoot connectivity by checking NSGs and effective routes first, then Network Watcher tools.

Securing Access to Virtual Networks

WHAT YOU'LL LEARN

- Create and configure network security groups (NSGs) and application security groups
- Evaluate effective security rules in NSGs
- Implement Azure Bastion
- Configure service endpoints for Azure PaaS
- Configure private endpoints for Azure PaaS

IN PLAIN ENGLISH

Building a network in Chapter 11 was like constructing a building. This chapter is about deciding who gets to walk through which doors — the core question is always “should THIS traffic, from THIS source, reach THAT destination?”

12.1 Creating NSGs and application security groups

DEFINITION

Network security group (NSG): A set of inbound and outbound security rules that allow or deny network traffic to subnets and network interfaces, based on source, destination, port, and protocol.

Each rule has a **priority** number; rules are evaluated lowest-first, and the first match wins. Traffic to a VM must pass **both** the subnet NSG and the NIC NSG if both exist — the more restrictive result wins.

Default rule	Priority	Direction	Effect
AllowVNetInBound	65000	Inbound	Allow all traffic from the VNet (incl. peered VNets)
AllowAzureLoadBalancerInBound	65001	Inbound	Allow Azure Load Balancer health probes
DenyAllInBound	65500	Inbound	Deny everything else
AllowVNetOutBound	65000	Outbound	Allow all traffic to the VNet
AllowInternetOutBound	65001	Outbound	Allow outbound traffic to the internet
DenyAllOutBound	65500	Outbound	Deny everything else

MEMORIZE THIS

Default rules sit at priorities **65000–65500** — outside your custom 100–4096 range, so any rule you create outranks them. **AllowInternetOutBound** is a **DEFAULT** rule — every VM can reach the internet unless you override it.

Application security groups

An **ASG** lets you group VMs by role (“web,” “db”) and write NSG rules that reference the group name instead of IPs — rules keep working as VMs come and go.

EXAM TIP

An NSG allows up to **1,000 rules**. **Augmented security rules** let a single rule hold up to **4,000 IP addresses/ranges or ports**, collapsing near-duplicate rules into one.

12.2 Evaluating effective security rules in NSGs

Because a subnet NSG and a NIC NSG can both apply, and default rules sit underneath your custom ones, the rule that actually decides a packet's fate isn't always obvious from reading one NSG alone. Use **effective security rules** to see the real, combined outcome instead of guessing.

COMMAND EXAMPLE

```
# View effective security rules for a NIC
az network nic list-effective-nsg --name myNic --resource-group myRG
```

MEMORIZE THIS

Effective security rules (portal: NIC → Networking → Effective security rules) merges subnet NSG + NIC NSG + defaults into the single answer that actually applies — this is the tool, not manual rule-reading, whenever a question asks you to “evaluate” what's allowed.

12.3 Implementing Azure Bastion

DEFINITION

Azure Bastion: A fully managed service that provides secure RDP/SSH access to your VMs directly through the Azure portal over TLS, without giving the VM a public IP or opening any inbound management ports to the internet.

ANALOGY

Bastion is a secure internal corridor to reach a room, so you never have to prop open an exterior door (a public IP) just to get inside.

MEMORIZE THIS

Azure Bastion = browser-based RDP/SSH with **no public IP** on the VM. Deployed into a dedicated subnet that must be named exactly **AzureBastionSubnet** (case-sensitive), minimum /26.

12.4 Configuring service endpoints for Azure PaaS

DEFINITION

Service endpoint: Routes VNet traffic to a PaaS service (like a storage account) over the Azure backbone; the service still has a public IP but can be restricted to accept traffic only from your VNet.

No DNS changes are needed — the service keeps its normal public hostname, just secured to accept only your VNet's traffic.

12.5 Configuring private endpoints for Azure PaaS

DEFINITION

Private endpoint: Gives a PaaS service a private IP address inside your VNet; traffic never touches the public internet at all.

A private endpoint needs a matching **private DNS zone** linked to your VNet so the service's normal public hostname resolves to the new private IP instead of the public one — without that DNS piece, clients still resolve to the public endpoint even though the private one exists.

MEMORIZE THIS

Service endpoint = secured route, still a public IP. **Private endpoint** = a **private IP** for the service inside your VNet (most secure, the usual “no public exposure” answer) — but requires linking a private DNS zone or clients keep resolving to the public address.

EXAM TIP

“We created a private endpoint but traffic still goes out over the public internet” almost always means a **missing or unlinked private DNS zone**.

12.6 Hands-on lab: reach a VM via Bastion and a storage account via private endpoint

- **Step 1.** In the VNet from Chapter 11's lab, add a subnet named exactly **AzureBastionSubnet** with a /26 prefix.
- **Step 2.** Deploy Azure Bastion (Basic SKU) into that subnet, with a new Standard public IP.
- **Step 3.** Once provisioned, select a VM in the VNet and click *Connect > Bastion* — sign in with the VM's credentials and confirm you get a remote session with zero public IP on the VM itself.
- **Step 4.** Create a storage account, then under *Networking* choose *Private endpoint connections > + Private endpoint*, target the **blob** sub-resource, and place it in a subnet in your VNet. Let Azure integrate the private DNS zone automatically.
- **Step 5.** From the VM (connected via Bastion), run `nslookup <account>.blob.core.windows.net` and confirm it resolves to a private 10.x address, not a public one.
- **Step 6.** Set the storage account's *Public network access* to Disabled, and confirm you can still reach it from the VM (via the private endpoint) but not from your own machine outside the VNet.
- **Step 7 (cleanup).** Delete the Bastion host first (it has an hourly cost), then the private endpoint and public IP if not needed further.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. Two NSG rules match the same traffic. Which applies?

A. The one with the lower priority number (first match wins).

Q2. What tool shows the real, combined outcome of subnet + NIC NSGs plus defaults?

A. Effective security rules.

Q3. How do you RDP to a VM without a public IP?

A. Use Azure Bastion.

Q4. Which gives a PaaS service a private IP inside your VNet?

A. A private endpoint.

Q5. A private endpoint exists but clients still resolve to the public address. What's missing?

A. A linked private DNS zone.

CHAPTER SUMMARY

KEY TAKEAWAYS

- NSGs allow/deny by priority (100–4096, lowest first, first match wins); defaults sit at 65000–65500.
- Effective security rules merge subnet + NIC NSGs + defaults into the real answer.
- Azure Bastion gives RDP/SSH with no public IP on the VM, via a dedicated AzureBastionSubnet.
- Service endpoints secure the route to PaaS (still public IP); private endpoints give it a private IP (needs DNS linking).

Name Resolution & Load Balancing

WHAT YOU'LL LEARN

- Configure Azure DNS
- Configure an internal or public load balancer
- Troubleshoot load balancing

IN PLAIN ENGLISH

Two ordinary questions that need special tools at cloud scale: “when someone types a website name, how does that turn into the right server?” (DNS) and “if I have five identical servers, how do visitors get spread across all of them?” (load balancing).

13.1 Configuring Azure DNS

Azure DNS hosts your public domains on Azure's name servers, while **private DNS zones** provide name resolution *inside* your VNets — including automatic registration of VM names — without exposing anything publicly.

MEMORIZE THIS

Azure DNS hosts public domains. **Private DNS zones** resolve names within your VNets and can auto-register VM records — the same private DNS zone mechanism that private endpoints (Chapter 12) rely on.

13.2 Configuring an internal or public load balancer

DEFINITION

Azure Load Balancer: A Layer-4 (TCP/UDP) load balancer that distributes incoming traffic across a backend pool of VMs for high availability and scale.

Can be **public** (internet-facing) or **internal** (private, for traffic inside your network). Health probes decide which backends are healthy.

Term	Meaning	
Frontend IP	Where traffic arrives (public or private)	
Backend pool	The VMs that receive the traffic	
Health probe	Checks backend health; unhealthy ones are removed from rotation	
Load-balancing rule	Maps a frontend port to a backend port	
	Basic Load Balancer	Standard Load Balancer
Backend pool	Availability set or single VMs	Any VM, VMSS, or zone-spanning setup
Availability Zone support	No	Yes
Security	Open by default	Secure by default — needs an explicit NSG to allow traffic
SLA	None	99.99%

MEMORIZE THIS

Standard Load Balancer is secure-by-default (needs an NSG to allow traffic through) while **Basic is open** — the opposite of what people often assume. Standard is required for any zone-redundant design.

Traffic Manager — global DNS-based routing

Method	Routes based on
Priority	A primary endpoint, with others as failover order
Weighted	A distribution ratio you assign across endpoints
Performance	Lowest network latency to the user
Geographic	The user's geographic location, for data-residency/compliance routing
Multivalued	Returns several healthy endpoint IPs at once
Subnet	The user's source IP range, mapped to a specific endpoint

EXAM TIP

Geographic routing is the answer when a question mentions data residency or “users in the EU must be served from EU endpoints only.” Performance routing optimizes speed, not compliance — a common wrong answer for residency scenarios.

13.3 Troubleshooting load balancing

When a backend isn't receiving traffic, the classic cause is a **failing health probe** — an unhealthy backend gets pulled from rotation. Also check that the load-balancing rule's frontend/backend ports actually match what the app is listening on, and that the NSG (Standard SKU) allows the probe and data traffic through.

MEMORIZE THIS

For load-balancer problems, a **failing health probe** is the classic first cause to check — the backend gets silently pulled from rotation with no error, just no traffic reaching it.

13.4 Hands-on lab: build a load-balanced pair with a health probe

- **Step 1.** Create two small VMs in the same VNet, each running a basic web server (e.g. the Azure IIS/Nginx quickstart extension) so each responds on port 80.
- **Step 2.** Create a **Standard** SKU public Load Balancer, add both VMs to its backend pool, and create a health probe on TCP port 80.
- **Step 3.** Add a load-balancing rule mapping frontend port 80 to backend port 80, referencing the health probe from step 2.
- **Step 4.** Browse to the load balancer's public IP several times (or refresh repeatedly) and confirm responses come from both VMs (e.g. by having each serve a page showing its own hostname).
- **Step 5.** Stop the web server process on one VM (without stopping the VM itself) and confirm within a minute or two that the load balancer stops sending it traffic — the health probe caught it.
- **Step 6.** Check the NSG on the backend subnet and confirm it allows inbound from **AzureLoadBalancer** — remove that rule temporarily and observe the health probe fail for BOTH VMs even though the web servers are fine, proving Standard SKU's secure-by-default behavior.
- **Step 7 (cleanup).** Restore the NSG rule, then delete the load balancer and VMs if not needed further.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. Which service resolves names privately inside your VNets and can auto-register VMs?

A. Private DNS zones.

Q2. At which layer does Azure Load Balancer operate?

A. Layer 4 (TCP/UDP).

Q3. You need URL-based routing and a web application firewall. Which service?

A. Application Gateway (Layer 7) — not covered by this chapter's objectives but worth knowing exists alongside Load Balancer.

Q4. VMs behind a load balancer get no traffic. What's the classic first thing to check?

A. The health probe (a failing probe removes backends from rotation).

Q5. Which Traffic Manager method fits 'EU users must be served only from EU endpoints'?

A. Geographic routing.

CHAPTER SUMMARY

KEY TAKEAWAYS

- Azure DNS hosts public domains; private DNS zones resolve names inside VNets.
- Azure Load Balancer is Layer 4; Standard SKU is secure-by-default and required for zone redundancy.
- Traffic Manager offers 6 global routing methods; Geographic fits residency/compliance requirements.
- Troubleshoot load balancing by checking the health probe first, then rule port mapping and NSG.

PART 5

Monitoring & Maintenance

Monitoring with Azure Monitor

WHAT YOU'LL LEARN

- Interpret metrics in Azure Monitor
- Configure log settings in Azure Monitor
- Query and analyze logs in Azure Monitor
- Set up alert rules, action groups, and alert processing rules
- Configure and interpret monitoring via Azure Monitor Insights
- Use Azure Network Watcher and Connection monitor

IN PLAIN ENGLISH

Once resources are running, a new question replaces “how do I build this?” with “is everything actually okay, and how would I know if it wasn't?” This chapter follows Microsoft's order exactly: first the raw signals (metrics, then logs), then reacting to them (alerts), then the pre-built dashboards (Monitor Insights) built on top of all of it, then the specialized network diagnostics tool last.

14.1 Interpreting metrics in Azure Monitor

Azure Monitor is the umbrella for observing everything in Azure. It collects two fundamentally different kinds of data.

	Metrics	Logs
What they are	Numeric values sampled over time (CPU %, disk IOPS)	Detailed, timestamped event records
Speed	Lightweight, near real-time	Richer but heavier to query
Stored in	The metrics database	A Log Analytics workspace
Queried with	Metrics explorer / charts	KQL (Kusto Query Language)

ANALOGY

Metrics are the speedometer — a constant numeric readout, great for instant alerts. Logs are the trip diary — detailed entries you search later to reconstruct what happened.

MEMORIZE THIS

Metrics = numeric, near-real-time, ideal for fast alerting, retained **93 days** by default in the metrics store itself.

14.2 Configuring log settings in Azure Monitor

By default, resource logs aren't collected anywhere queryable. **Diagnostic settings** configure what gets collected and where it's sent — a Log Analytics workspace (for KQL, see 14.3), a storage account (cheap long-term archive), or an Event Hub (streaming to a SIEM or third-party tool).

COMMAND EXAMPLE

```
# Route a VM's metrics/logs to a Log Analytics workspace
az monitor diagnostic-settings create --name vm-diag \
  --resource <resource> --workspace <workspace> \
  --metrics '[{"category": "AllMetrics", "enabled": true}]'
```

Data type	Default retention	Configurable range
Log Analytics workspace logs	30 days (free/basic)	Up to 730 days (2 years) per table, at added cost
Activity log	90 days (kept automatically)	Export for longer via diagnostic settings

MEMORIZE THIS

If a scenario needs log data kept longer than the default window, the answer is “**configure log settings (diagnostic settings) to send it to a Log Analytics workspace** (or storage account for cheap archive).” Nothing is retained indefinitely without explicit configuration.

14.3 Querying and analyzing logs in Azure Monitor

Once logs land in a Log Analytics workspace (via 14.2's diagnostic settings), you query them with **KQL**.

KQL EXAMPLE

```
# Find VMs with the most restart events in the last 24 hours
Event
| where TimeGenerated > ago(24h)
| where EventID == 6008
| summarize RestartCount = count() by Computer
| order by RestartCount desc
```

MEMORIZE THIS

KQL basics: **where** filters, **summarize** aggregates, **order by** sorts. Every table has a **TimeGenerated** column — always filter on it first for performance.

14.4 Setting up alert rules, action groups, and processing rules

Component	Role
Alert rule	The condition that triggers an alert (metric, log, or activity-log based)
Action group	The set of actions taken when it fires: email/SMS, webhook, Logic App, Automation runbook
Alert processing rule	Applies actions or suppression to alerts in bulk (e.g. mute during a maintenance window)

MEMORIZE THIS

Alert rule = the trigger condition; **action group** = who/what responds; **alert processing rules** suppress/modify alerts in bulk, e.g. during planned maintenance.

14.5 Configuring and interpreting Azure Monitor Insights

Insights are pre-built, curated monitoring experiences layered on top of the metrics and logs you've already configured — VM Insights, Storage Insights, Network Insights — giving ready-made dashboards (dependency maps, performance charts) without hand-building queries yourself.

MEMORIZE THIS

Monitor Insights require the underlying data collection (14.1–14.3) to already be flowing — they're a curated VIEW on top of your metrics/logs, not a separate data source. VM Insights specifically needs the Azure Monitor Agent installed.

14.6 Using Azure Network Watcher and Connection monitor

Tool	Answers
IP flow verify	"Is this specific traffic allowed or denied, and by which rule?" — one-time check
NSG diagnostics	Which NSG/rule decided the outcome, across a chain of NSGs
Connection troubleshoot	"Can A reach B right now?" — tests the full path end to end
Connection Monitor	The same test, running continuously over time with alerting

EXAM TIP

A one-time "why is this blocked" question points to **IP flow verify**. An ongoing "alert us if connectivity degrades" requirement points to **Connection Monitor**. Network Watcher is covered LAST in this chapter because it's the specialized network-diagnostics layer, distinct from general Azure Monitor.

14.7 Hands-on lab: wire up a metric alert end to end

- **Step 1.** Create a Log Analytics workspace, then a diagnostic setting on one of your test VMs sending **AllMetrics** to it (this is 14.2 in practice).
- **Step 2.** In *Monitor > Logs*, run the KQL query from 14.3 (or a simpler **Perf | take 10** if Event isn't populated yet) against your workspace to confirm data is actually flowing.
- **Step 3.** Create an action group (*Monitor > Alerts > Action groups > Create*) with an email notification pointed at your own address.
- **Step 4.** Create a metric alert rule on the VM: condition = Percentage CPU greater than 1 (deliberately low so it fires easily for this lab), evaluated every 1 minute, linked to your action group.
- **Step 5.** Confirm you receive the alert email within a few minutes, then check *Monitor > Alerts* in the portal and confirm the fired alert is listed there too.
- **Step 6.** Enable *VM Insights* on the same VM (*Monitor > Insights > Virtual Machines*) and confirm the pre-built performance dashboard populates — this is 14.5 built on top of the data you already wired up.
- **Step 7 (cleanup).** Delete or disable the alert rule (a 1% threshold will otherwise fire constantly and spam your inbox).

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. Which data type is best for a fast alert on high CPU: metrics or logs?

A. Metrics (numeric, near-real-time).

Q2. What language queries logs in a Log Analytics workspace?

A. KQL (Kusto Query Language).

Q3. An alert fires — what defines whether it emails you or runs a runbook?

A. The action group.

Q4. Do Monitor Insights collect their own separate data, or build on what you've already configured?

A. They build on the metrics/logs you've already configured — a curated view, not a separate source.

Q5. Which Network Watcher tool tells you if an NSG is blocking specific traffic right now?

A. IP flow verify.

CHAPTER SUMMARY

KEY TAKEAWAYS

- Metrics (numeric, fast, 93-day default retention) vs logs (rich, KQL, Log Analytics workspace).
- Diagnostic settings (log settings) route data to a workspace/storage account/Event Hub; nothing is retained by default without this.
- Alert rules trigger; action groups respond; alert processing rules modify/suppress alerts in bulk.
- Monitor Insights are curated dashboards built on your existing metrics/logs, not a separate data source.
- Network Watcher (IP flow verify, next hop, Connection Monitor) is the specialized network-diagnostics layer, covered last.

Backup & Recovery

WHAT YOU'LL LEARN

- Create a Recovery Services vault
- Create an Azure Backup vault
- Create and configure a backup policy
- Perform backup and restore operations by using Azure Backup
- Configure Azure Site Recovery for Azure resources
- Perform a failover to a secondary region by using Site Recovery
- Configure and interpret reports and alerts for backups

IN PLAIN ENGLISH

This last chapter is about what happens when things go wrong — someone deletes the wrong file, ransomware hits a server, or a whole region goes offline. Two questions only: “how do I get a copy of my data back?” (backup) and “how do I keep the service running somewhere else if the primary location is gone?” (disaster recovery).

15.1 Creating a Recovery Services vault

DEFINITION

Recovery Services vault: The storage container for backup data (VMs, files, SQL in VMs) and for Azure Site Recovery replication configuration — the classic, general-purpose vault type.

COMMAND EXAMPLE

```
# Create a Recovery Services vault
az backup vault create --resource-group myRG --name myVault --location eastus
```

15.2 Creating an Azure Backup vault

A newer, separate vault type used for certain workloads (like Azure Database for PostgreSQL and disk-level backups) that use Azure Backup’s newer vaulted-backup architecture rather than the classic Recovery Services vault.

MEMORIZE THIS

Recovery Services vault = the classic, general-purpose vault (VMs, files, SQL in VMs, and Site Recovery). **Azure Backup vault** = the newer vault type for specific workloads using vaulted backup. Know both exist — which one a workload uses isn’t always your choice.

15.3 Creating and configuring a backup policy

A backup policy is a layered retention schedule, not just “daily or weekly.”

Policy layer	Typical example
Backup frequency	Once daily (VM backup) or more often for some workloads
Retention: daily points	Keep each daily recovery point for, say, 30 days
Retention: weekly points	Keep one recovery point per week for, say, 12 weeks

Policy layer	Typical example
Retention: monthly points	Keep one per month for, say, 12 months
Retention: yearly points	Keep one per year for, say, 5–10 years

MEMORIZE THIS

These layers **stack** — a single recovery point can simultaneously count as this week's weekly point AND this month's monthly point, so you don't need separate backup jobs per retention tier.

15.4 Performing backup and restore operations

COMMAND EXAMPLE

```
# Enable backup for a VM into a Recovery Services vault
az backup protection enable-for-vm --resource-group myRG \
  --vault-name myVault --vm myVM --policy-name DefaultPolicy
```

Restoring can create a new VM from a recovery point, restore just disks, or (for file-level restore) mount the recovery point and copy out individual files without a full VM restore.

15.5 Configuring Azure Site Recovery

DEFINITION

Azure Site Recovery (ASR): A disaster-recovery service that continuously replicates whole workloads (VMs) to another region, so you can fail over and keep running if the primary region goes down.

ANALOGY

Backup is a photo album — snapshots you restore from after something is lost. Site Recovery is a stunt double standing by in another city — if the primary goes down mid-show, the double steps in.

15.6 Performing a failover to a secondary region

Failover switches active workloads to the replicated copy in the secondary region. This can be a planned, tested failover (no data loss, used for drills) or an unplanned failover after a real outage (may lose data since the last replication cycle — this is the RPO in practice).

MEMORIZE THIS

Azure Backup = restore data from recovery points. **Azure Site Recovery = replicate and fail over** whole workloads to another region for business continuity. If the question is about recovering deleted/corrupted data → Backup. If it's about surviving a whole region outage with minimal downtime → Site Recovery.

15.7 Configuring and interpreting reports and alerts for backups

Backup reporting surfaces job success/failure trends, storage consumption, and compliance against policy across all your vaults — and backup alerts notify you when a scheduled job fails, so a silent backup failure doesn't go unnoticed until you actually need to restore.

RPO and RTO

Term	Means	Addressed by
RPO (Recovery Point Objective)	Acceptable data loss (in time) — “we can lose up to 1 hour”	Backup frequency / replication frequency

Term	Means	Addressed by
RTO (Recovery Time Objective)	Acceptable downtime — “we must be back within 4 hours”	Site Recovery's failover speed; Backup's restore speed

EXAM TIP

A maximum acceptable **data loss** window describes **RPO**. A maximum acceptable **downtime** window describes **RTO**. A backup failure alert that goes unnoticed is exactly the kind of gap that blows your RPO without anyone knowing until restore time — which is why 15.7 (reporting/alerts) matters as much as the policy itself.

15.8 Hands-on lab: back up a VM and test a file-level restore

- **Step 1.** Create a Recovery Services vault, then enable backup on one of your test VMs using the default policy.
- **Step 2.** Trigger an on-demand backup (don't wait for the schedule): **Backup Center > select the VM > Backup now**.
- **Step 3.** Once the job completes, go to the VM's *Backup* blade and choose *File Recovery* — download the provided script/executable that mounts the recovery point as a local drive.
- **Step 4.** Run it on the VM (via Bastion from Chapter 12's lab) and confirm you can browse and copy out individual files without restoring the whole VM.
- **Step 5.** Go to *Backup reports* at the vault level and confirm your on-demand job shows up in the job history with its status.
- **Step 6.** Create a backup alert rule (or check the vault's built-in alert settings) for backup job failures, and confirm an action group is attached to it.
- **Step 7 (cleanup).** Stop protection (with or without retaining existing recovery points, your choice) once you're satisfied, to avoid ongoing backup storage costs.

QUICK CHECK

Cover the answers. Say your answer out loud, then reveal.

Q1. Where are VM backups stored, and what defines their schedule and retention?

A. A Recovery Services vault; a backup policy.

Q2. Ransomware encrypted a VM's data. Which service restores it?

A. Azure Backup (restore from a clean recovery point).

Q3. An entire Azure region goes offline. Which service keeps your workload running elsewhere?

A. Azure Site Recovery (fail over to the secondary region).

Q4. A scenario states 'we can tolerate at most 1 hour of data loss.' RPO or RTO?

A. RPO — it's about acceptable data loss, not downtime.

Q5. Why do backup reports and alerts matter as much as the policy itself?

A. A silently failing backup job can blow your RPO without anyone noticing until you actually try to restore.

CHAPTER SUMMARY

KEY TAKEAWAYS

- Recovery Services vault (classic, general-purpose) vs Azure Backup vault (newer, specific workloads).
- Backup policies layer daily/weekly/monthly/yearly retention that stack on shared recovery points.
- Azure Backup restores data from recovery points; Site Recovery replicates and fails over whole workloads.
- RPO = acceptable data loss window; RTO = acceptable downtime window.
- Backup reports and alerts catch silent job failures before they become a real RPO problem.

PART T

Troubleshooting Guide

What Works, What Doesn't: A Troubleshooting Reference

Every domain of the exam, distilled into the specific symptom you'd actually see, the real cause behind it, and the fix — organized in the same five-part order as the rest of this book. Use this as a pre-exam sweep, or as a real-world quick reference once you're on the job.

Domain 1 — Identities & Governance

Symptom	Likely cause	Fix
A dynamic group's membership doesn't reflect a recent attribute change	Dynamic membership evaluation isn't instant	Wait for the next evaluation cycle; don't assume the rule failed
A user has two licenses but a service (e.g. Exchange) still doesn't work	A duplicate/conflicting service plan was auto-disabled, OR usage location isn't set	Check the license's service plan list for a disabled entry; confirm Usage location is set on the user
A custom role's NotActions exclusion doesn't seem to actually block anything	NotActions is a subtraction from ONE role definition, not a tenant-wide Deny	Check whether the user has a different, second role assignment granting that same permission
A resource can't be deleted even though the user is Owner	A resource lock (CanNotDelete) is applied, possibly inherited from the resource group	Check Locks at both the resource and its parent resource group/subscription
A new resource keeps getting blocked on creation with a policy violation	An Azure Policy assignment with a Deny effect is scoped here	Check Policy > Compliance, find the assignment, and either fix the resource or request an exemption
A tag added to a resource group doesn't appear on the resources inside it	Tags are not inherited by default	Use a Policy with the Modify or Append effect to force tag propagation, or tag each resource directly

Domain 2 — Storage

Symptom	Likely cause	Fix
A SAS token that should still be valid no longer works	It was tied to a stored access policy that was modified or deleted	Check the container's stored access policies; reissue the SAS if the policy was intentionally revoked
An app can't reach a storage account it could reach yesterday	The storage firewall's default action is now Deny and the app's network isn't on the allow list	Add the app's VNet/subnet or IP range to the storage account's Networking allow list
A blob you thought was deleted is still incurring storage costs	Soft delete is retaining it for the configured retention window (this is expected, not a bug)	Wait out the retention window, or use Content > Show deleted blobs to permanently purge it if truly not needed
Moving a blob out of Archive costs more than expected	Blobs have minimum-stay periods (Cool 30d / Cold 90d / Archive 180d); moving early triggers an early-deletion fee	This is billed by design — plan lifecycle rules around the minimum stay periods
An Archive-tier blob won't open immediately	Archive is offline by design and must be rehydrated first	Set rehydration priority to High for urgent same-hour needs; Standard can take up to ~15 hours
Identity-based access to an Azure Files share is denied even with the right RBAC role	NTFS-style permissions on the files/folders are a second, independent layer	Confirm both the Azure RBAC role AND the NTFS permission allow the action — both must agree

Domain 3 — Compute

Symptom	Likely cause	Fix
Redeploying a Bicep/ARM template unexpectedly deletes unrelated resources	The deployment was run in Complete mode instead of the default Incremental	Always confirm deployment mode; use what-if before any Complete-mode deployment
A VM can't be added to an existing availability set	Availability set membership can only be set at VM creation time	Redeploy the VM into the set from the start; there's no after-the-fact fix
A VM's SLA doesn't match what was expected for its configuration	Not every disk on the VM is Premium SSD/Ultra — the 99.9% single-VM SLA requires ALL disks to qualify	Upgrade every attached disk (OS and data) to a qualifying tier
Enabling encryption at host fails immediately	The EncryptionAtHost feature isn't registered on the subscription yet	Run az feature register for Microsoft.Compute/EncryptionAtHost and wait for it to show Registered
A container in ACI keeps restarting in an unwanted loop	The restart policy is set to Always (the default) on what was meant to be a one-off job	Set the restart policy to Never or OnFailure for batch-style jobs
A deployment slot swap didn't bring over an expected setting	The setting is marked as a deployment slot setting, which deliberately stays with the slot	This is expected behavior — uncheck the slot-setting flag if the value SHOULD travel with a swap

Domain 4 — Networking

Symptom	Likely cause	Fix
Two peered VNets can't reach a third VNet peered to one of them	VNet peering is not transitive	Create a direct peering to the third VNet, or route through a hub with a gateway/NVA
A subnet can't reach the internet even though the NSG allows it	A user-defined route is redirecting traffic elsewhere (e.g. next hop = a firewall, or None)	Check Effective routes on the NIC; adjust or remove the UDR
Traffic is unexpectedly blocked and the NSG rules look correct	A DIFFERENT NSG (subnet vs NIC) is also in play, or a default rule is winning	Check Effective security rules on the NIC — it merges both NSGs and defaults into the real answer
Azure Bastion deployment fails	The AzureBastionSubnet is missing, misnamed, or smaller than /26	Create a subnet named exactly AzureBastionSubnet (case-sensitive) at /26 or larger
A private endpoint exists but clients still resolve to the public IP	No private DNS zone is linked to the VNet, or it isn't linked at all	Create/link a privatelink DNS zone for that service and confirm the VNet link is present
A Standard SKU load balancer's backends never receive traffic	Standard Load Balancer is secure-by-default; no NSG rule allows AzureLoadBalancer or the data traffic	Add explicit NSG rules allowing AzureLoadBalancer inbound and the actual app traffic

Domain 5 — Monitoring & Maintenance

Symptom	Likely cause	Fix
A KQL query against a workspace returns nothing	No diagnostic setting is routing that resource's logs to the workspace yet	Configure log settings (diagnostic settings) on the resource, pointed at the workspace
Log data older than expected is missing	Default Log Analytics retention is 30 days unless explicitly extended	Increase retention on the workspace/table (up to 730 days) if longer history is required
An alert rule never fires even though the condition is clearly met	The action group isn't attached, or the evaluation frequency/window doesn't match the spike's duration	Confirm the action group is linked; shorten the evaluation window if spikes are brief
VM Insights shows no data after being enabled	The Azure Monitor Agent isn't installed/running on the VM yet	Confirm the AMA extension status on the VM; reinstall if it shows as failed

Symptom	Likely cause	Fix
A load-balanced VM silently stops receiving traffic	Its health probe is failing, even though the app process looks fine from inside the VM	Check what port/path the probe actually tests, and confirm the app responds on exactly that
A restore from Azure Backup fails or is unavailable	The scheduled backup job itself failed silently and there's no valid recent recovery point	Set up backup alerts/reports (Chapter 15.7) so a failed job is caught before you need to restore, not after

MEMORIZE THIS

Notice the pattern across every domain: the exam's favorite "broken" scenario is almost always a **default behavior nobody explicitly changed** (a minimum stay period, a secure-by-default SKU, a non-transitive peering) rather than an outright misconfiguration. Knowing the defaults cold is what separates "works" from "why doesn't this work."

PART P

Case Study Practice

Case Study Practice

The real exam presents a company background once, then asks several questions against that SAME scenario — forcing you to combine knowledge across domains instead of answering one isolated fact at a time. These two case studies do the same, and deliberately mix question formats: standard single-answer, multi-select (“select all that apply”), and a sequencing/ordering question, matching the variety the real exam uses.

Case Study 1: Contoso Manufacturing

Background

Contoso Manufacturing has 400 employees across a head office and two regional plants. They run an on-premises Active Directory domain and want to extend identity to Azure without abandoning their existing AD. IT is a small, centralized team, but each regional plant has a local IT contact who should be able to reset passwords and manage devices for their own plant only — nothing else.

Contoso is deploying a new internal web application on Azure App Service, backed by a database server running on a VM. The web app must be reachable from the internet; the database VM must never be directly reachable from the internet, only from the web tier and from IT's jump host.

Finance requires monthly reports showing spend per plant, and wants a hard stop if any plant's monthly Azure spend is projected to exceed budget significantly. Contoso also wants alerts if the database VM's CPU spikes, since that has preceded past outages.

Contoso must be able to recover the database VM to a point earlier that day if data is corrupted, and separately must be able to keep the application running from a secondary region if the primary region has an outage.

Question 1. Which hybrid identity approach lets Contoso keep on-prem AD as the source of truth while extending identity to Azure, with the lowest operational overhead?

Answer. Password hash synchronization (PHS) via Microsoft Entra Cloud Sync — simplest to deploy and operate, cloud-only authentication, and enables leaked-credential detection. Pass-through authentication would be justified only if policy required credentials never leave on-prem, which isn't stated here.

Question 2. What should Contoso use to let each regional plant's local IT contact reset passwords and manage devices for their own plant ONLY, without granting tenant-wide admin rights? (Select all that apply.)

Answer. An administrative unit (AU) scoped to each plant's users/devices, AND a Helpdesk Administrator (or similar least-privileged) role assigned AT that AU rather than tenant-wide. Both pieces are required together — the AU alone doesn't grant any rights, and the role alone (without AU scoping) would be tenant-wide.

Question 3. How should the database VM's network access be configured so it's reachable from the web tier and IT's jump host, but never directly from the internet?

Answer. No public IP on the database VM; an NSG on its subnet allowing inbound only from the web tier's subnet and from the jump host's subnet/Bastion; administrative access via Azure Bastion rather than a public RDP/SSH port.

Question 4. Which Azure Monitor components does Contoso need to combine to (a) alert when the database VM's CPU spikes and (b) get a hard stop before a plant's spend runs away? (Select all that apply.)

Answer. (a) A metric alert rule on the VM's CPU percentage, linked to an action group for notification — this is Azure Monitor. (b) A budget with an alert threshold in Cost Management. Note that a budget only notifies; it does not automatically stop resources, so “hard stop” in the real world still requires a human or an automated runbook triggered by the budget alert — Azure has no native auto-shutoff on budget breach.

Question 5. Which service addresses “recover the database VM to a point earlier today,” and which separate service addresses “keep the application running from a secondary region during an outage”?

Answer. Azure Backup (restore from an earlier recovery point) for the first; Azure Site Recovery (replicate and fail over to a secondary region) for the second. These are two different services solving two different problems, both needed here.

Question 6. Put these steps in the correct order to stand up the database VM's network isolation described above: (A) Create the NSG rule allowing the web subnet, (B) Create the VNet and subnets, (C) Create the VM with no public IP in the correct subnet, (D) Deploy Azure Bastion into a dedicated subnet.

Answer. B, D, C, A — the network and its Bastion subnet must exist before you can place a VM into it without a public IP, and an NSG rule references a subnet that must already exist. (In practice several of these can be done in one Bicep deployment, but the logical dependency order is still B before C, and D can happen anytime before you need Bastion access.)

Case Study 2: Fabrikam Health

Background

Fabrikam Health stores patient records in Azure Blob Storage and must ensure that storage account is never reachable from the public internet, under any circumstances — only from Fabrikam's own VNet.

Auditors require that Fabrikam can show, for the last 90 days, every time a specific admin role was activated and by whom, without permanent standing administrator accounts existing in the tenant.

Patient record files older than 2 years are rarely accessed but must be retained for 10 years for compliance, and Fabrikam wants to minimize storage cost for that older data without risking accidental permanent loss of recent files.

Fabrikam's application team wants a new internal tool published without exposing it directly to the internet or requiring a VPN client on every laptop.

Question 1. How should the storage account be configured to guarantee it's never reachable from the public internet?

Answer. Set the storage account's public network access to Disabled, and provide access exclusively through a private endpoint into Fabrikam's VNet, with a linked private DNS zone so the normal hostname resolves to the private IP. A storage firewall alone (allowing specific IPs) still leaves the account reachable via the public endpoint from allowed sources — only disabling public access entirely plus a private endpoint fully satisfies “never reachable from the internet.”

Question 2. Which Azure capability satisfies the audit requirement (who activated a privileged role, when, for the last 90 days) while eliminating standing admin accounts?

Answer. Microsoft Entra Privileged Identity Management (PIM), with roles set to eligible rather than permanently active, activation requiring justification/MFA, and PIM audit history reviewed for the reporting window.

Question 3. Which blob storage features together minimize cost for old records while protecting against accidental loss of RECENT files? (Select all that apply.)

Answer. A lifecycle management rule moving blobs untouched for 2 years to the Archive tier (cheapest storage for rarely-accessed data), AND soft delete plus versioning enabled to protect recent, actively-used files from accidental deletion or overwrite. Archive tier alone doesn't protect against accidental deletion of recent files still in Hot/Cool.

Question 4. What's the correct answer for publishing the internal tool without internet exposure or a VPN client requirement — and why is a public IP with an NSG NOT the best answer here?

Answer. If the tool is a web app reachable via HTTP(S) and the requirement is no direct internet exposure, Microsoft Entra Application Proxy publishes on-prem/internal web apps through Entra ID with no inbound firewall ports opened at all. A public IP + NSG would still expose SOME internet-facing surface and doesn't provide the identity-aware access control Application Proxy adds — it directly fails the “not exposed to the internet” requirement.

Question 5. A 3-year-old patient record file needs to be retrieved for an urgent legal request within the hour. What must happen, and is the timing guaranteed?

Answer. The file is in the Archive tier (per the lifecycle rule) and is offline — it must be rehydrated before it can be read. Request High-priority rehydration for same-hour retrieval, but note this is best-effort under 1 hour for blobs under 10 GB, not a guaranteed SLA.

Question 6. True or False, with justification: “Because PIM is configured, Fabrikam no longer needs a break-glass emergency access account.”

Answer. False. Break-glass accounts exist specifically for when normal sign-in, Conditional Access, or PIM itself is unavailable or misconfigured — they are deliberately excluded from Conditional Access and kept permanently active, which is the opposite of how PIM-managed roles work. PIM reduces reliance on standing admin accounts for DAY-TO-DAY use; it doesn't replace the emergency-recovery mechanism.

MEMORIZE THIS

Case studies reward the same skill as troubleshooting: identify the MOST SPECIFIC constraint in the background (“never reachable from the internet,” “no standing admin accounts,” “no VPN client”) and let it eliminate the plausible-but-incomplete answers. A partially-correct answer that misses one stated constraint is a wrong answer on the real exam.

PART A

Appendix

Appendix A · Glossary

Quick definitions for the terms AZ-104 expects you to know, in the order they first appear.

Term	Meaning
Management group	Container for many subscriptions; applies governance at scale (covered last in Ch 3, widest scope).
Subscription	Billing and access boundary for resources.
Resource group	Folder holding related resources with a shared lifecycle.
Azure Resource Manager (ARM)	The management layer every request passes through.
Member vs guest user	Internal account vs external B2B collaborator (#EXT#).
Dynamic group	Membership set by an attribute rule, evaluated on a schedule (not instant); needs Entra ID P1.
SSPR	Self-service password reset (register methods first); combined registration covers MFA + SSPR.
Azure RBAC	Access to Azure resources via role assignments (principal + role + scope).
Owner / Contributor / Reader	Full+grant / manage-but-not-grant / view-only.
NotActions	Subtracts permissions from a custom role definition — not a Deny.
Entra role	Directory admin role (e.g. Global Administrator) — not Azure RBAC.
Azure Policy	Enforces/audits what resources are allowed (Deny/Audit/Append/Modify/DINE).
Resource lock	CanNotDelete or ReadOnly; inherited; overrides RBAC for delete.
SAS types	User delegation (Entra ID, most secure) / Service (one service) / Account (multiple services).
Stored access policy	Server-side control that lets you centrally revoke issued SAS tokens.
Redundancy	LRS/ZRS/GRS/GZRS; RA- adds read access to the secondary; GRS failover is manual.
Object replication	Deliberate account-to-account blob copy, distinct from automatic redundancy.
Access tier	Hot/Cool/Cold/Archive; Archive is offline (rehydrate: Standard ~15h, High <1h).
Soft delete / versioning	Recover deleted items / recover overwritten items.
Lifecycle management	Auto-tier or delete blobs by age/last access.
ARM template / Bicep	JSON IaC / friendlier IaC that transpiles to ARM; both idempotent.
Deployment mode	Incremental (default, safe) vs Complete (deletes extras).
Availability Set	Fault + update domains within a datacenter (~99.95%); set at creation only.
Availability Zones	Separate datacenters within a region (~99.99%).
VMSS	Virtual Machine Scale Set — identical auto-scaling VMs.
ACR / ACI / Container Apps	Image registry (Premium = geo-replication) / single container / autoscaling serverless containers (can scale to zero).
App Service / plan	PaaS web hosting / defines size, tier, cost, features (Standard+ for slots/autoscale).
Deployment slot	Staging instance swapped to prod for zero-downtime; slot-specific settings don't swap.
VNet / subnet	Private network / segment of it (Azure reserves 5 IPs/subnet).
VNet peering	Connects VNets over the backbone; non-transitive.
UDR	User-defined route; overrides default routing; next hop None deliberately blocks traffic.
NSG	Allow/deny rules; priority 100–4096, defaults at 65000–65500, lowest first, first match wins.
Effective security rules	The merged, real outcome of subnet NSG + NIC NSG + defaults.

Term	Meaning
Azure Bastion	RDP/SSH to VMs with no public IP; needs AzureBastionSubnet, /26 minimum.
Service / private endpoint	Secured route to PaaS (still public IP) / private IP for PaaS (needs linked private DNS zone).
Azure Load Balancer	Layer-4 (TCP/UDP); Standard SKU is secure-by-default and required for zone redundancy.
Traffic Manager	Global DNS-based routing; 6 methods including Geographic for residency requirements.
Metrics vs logs	Numeric near-real-time (93-day retention) / rich records queried with KQL (30-day default, up to 730).
Diagnostic settings (log settings)	Routes metrics/logs to a Log Analytics workspace, storage account, or Event Hub.
Alert rule / action group	Trigger condition / the response actions.
Monitor Insights	Curated dashboards built on your existing metrics/logs, not a separate data source.
Network Watcher	Network diagnostics (IP flow verify, next hop, Connection Monitor).
Recovery Services vault / Azure Backup vault	Classic general-purpose vault / newer vault for specific workloads.
Azure Site Recovery	Replicate and fail over workloads to another region.
RPO / RTO	Acceptable data loss window / acceptable downtime window.

Appendix B · Exam-Day Strategy

- **Do the labs early.** AZ-104 has interactive lab tasks in a live portal. This book's hands-on labs in every chapter are meant to be actually run, not just read.
- **Read the last sentence first** in long scenario questions to know what's actually asked.
- **Watch the near-twins:** Backup vs Site Recovery; Availability Set vs Zones; service vs private endpoint; scale up vs out; Azure RBAC vs Entra roles; Contributor vs Owner; NSG vs effective security rules; Incremental vs Complete deployment.
- **Recognize both syntaxes:** Azure CLI (`az ...`) and PowerShell (`Verb-AzNoun`).
- **Remember the defaults:** NSG priority lowest-first; Archive is offline; peering non-transitive; Contributor can't assign roles; tags don't inherit; deployment mode defaults to Incremental.
- **Use this book's order as your checklist.** Every chapter and section number matches the official study guide's own order — if you can recite what each numbered section covers, you've covered the guide completely.
- **Flag and move on.** Answer everything; no penalty for guessing.

EXAM TIP

Aim for 85%+ on Microsoft's free official AZ-104 practice assessment, and be able to perform each hands-on lab task once by hand in a real subscription, before you book.

Appendix C · CLI & PowerShell Quick Reference

Recognize the pattern: CLI is **az** <service> <action>; PowerShell is <Verb>-Az<Noun>.

Task	Azure CLI	PowerShell
Create resource group	az group create	New-AzResourceGroup
Create VM	az vm create	New-AzVM
List VMs	az vm list	Get-AzVM
Create storage account	az storage account create	New-AzStorageAccount
Create VNet	az network vnet create	New-AzVirtualNetwork
Create NSG	az network nsg create	New-AzNetworkSecurityGroup
Assign a role	az role assignment create	New-AzRoleAssignment
List role assignments	az role assignment list	Get-AzRoleAssignment
Deploy template	az deployment group create	New-AzResourceGroupDeployment
What-if a deployment	az deployment group what-if	(use Test- prefix equivalents)
View effective NSG rules	az network nic list-effective-nsg	Get-AzEffectiveNetworkSecurityGroup
Enable VM backup	az backup protection enable-for-vm	Enable-AzRecoveryServicesBackupProtection
Register a feature	az feature register	Register-AzProviderFeature

MEMORIZE THIS

CLI verbs: **create** / **list** / **show** / **delete** / **update**. PowerShell verbs: **New-** / **Get-** / **Set-** / **Remove-**. If you see **az** it's CLI; if you see **-Az** it's PowerShell.

Appendix D · Practice Question Bank

Thirty exam-style questions in the same order as the five parts. Cover the answer key and test yourself.

1. Create users and groups: what's the difference between a member and a guest user?
2. What must a user register before they can use SSPR?
3. Which built-in role can manage all resources but cannot assign roles to others?
4. You assign Reader at the subscription scope. What does the user see?
5. Is a custom role's NotActions the same as a Deny assignment?
6. You must ensure no VM larger than a certain size is ever created. RBAC or Azure Policy?
7. Which resource lock allows changes but prevents deletion?
8. Does a budget stop resources when exceeded?
9. Why is management groups covered last in the governance domain even though it's the top of the hierarchy?
10. Which SAS type is signed with Entra ID credentials instead of the account key?
11. What lets you centrally revoke SAS tokens that were already issued?
12. Which redundancy option is the cheapest that survives a full regional outage?
13. Is a GRS failover to the secondary region automatic?
14. Which tier is offline and must be rehydrated before reading?
15. What does Bicep compile to, and are deployments idempotent?
16. You deploy in Complete mode. What happens to resources in the RG not in the template?
17. Which protects VMs against an entire datacenter failure: availability set or zones?
18. Can an already-running VM be added to an availability set?
19. Does ACI scale to zero when idle?
20. What tier is required for App Service deployment slots?
21. VNet A peers B, B peers with C. Can A reach C?
22. What tool merges subnet + NIC NSGs and defaults into the real outcome?
23. What subnet name and size does Azure Bastion require?
24. A private endpoint exists but clients still resolve publicly. What's missing?
25. Which Load Balancer SKU is secure-by-default?
26. Metrics or logs for a fast CPU alert? What retains 93 days by default?
27. Do Monitor Insights collect their own data source?
28. Backup vs Site Recovery — which handles a full region outage?
29. A requirement states 'at most 1 hour of data loss.' RPO or RTO?
30. Why do backup alerts matter as much as the backup policy itself?

Answer Key

1. A member belongs to your tenant directly; a guest is external, invited via B2B (#EXT# in the UPN).
2. Their authentication methods (registered in advance).
3. Contributor.
4. Every resource group and resource in that subscription (inheritance).
5. No — it only subtracts from that one role definition; a different assignment can still grant the permission.
6. Azure Policy (Deny effect).
7. CanNotDelete.

8. No — it triggers alerts only.
9. Because it's the broadest, scale-out governance tool — Microsoft's guide covers narrower controls first, then the widest-reaching one last.
10. A user delegation SAS.
11. A stored access policy the SAS references.
12. GRS (geo-redundant).
13. No — it's a manual, customer-initiated action (or Microsoft-initiated only in a true disaster).
14. Archive.
15. ARM JSON; yes, idempotent.
16. They get deleted — Complete mode makes the resource group match the template exactly.
17. Availability Zones.
18. No — availability set placement can only be chosen at VM creation time.
19. No — only Container Apps scales to zero; ACI runs (and bills) until explicitly stopped.
20. Standard tier or higher.
21. No — peering is non-transitive.
22. Effective security rules.
23. AzureBastionSubnet (exact, case-sensitive name), /26 minimum.
24. A missing or unlinked private DNS zone.
25. Standard.
26. Metrics (fast, near-real-time); metrics retain 93 days by default in the metrics store.
27. No — they're a curated view built on metrics/logs you've already configured.
28. Azure Site Recovery (failover to secondary region).
29. RPO — it's about acceptable data loss, not downtime.
30. A silently failed backup job leaves no valid recovery point, which you only discover when you actually need to restore — blowing your RPO without warning.

MEMORIZE THIS

If you can answer all thirty cold, walk through every hands-on lab once in a real subscription, and explain every near-twin in Appendix B, you're ready to book AZ-104.